

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

DIPLOMSKI RAD br. 41

**SUSTAV ZA PRENOŠENJE 4K VIDEO SNIMKE S NISKOM
LATENCIJOM NA MALE UDALJENOSTI**

Matija Holik

Zagreb, lipanj 2023.

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

DIPLOMSKI RAD br. 41

**SUSTAV ZA PRENOŠENJE 4K VIDEO SNIMKE S NISKOM
LATENCIJOM NA MALE UDALJENOSTI**

Matija Holik

Zagreb, lipanj 2023.

Zagreb, 10. ožujka 2023.

DIPLOMSKI ZADATAK br. 41

Pristupnik: **Matija Holik (0036516800)**
Studij: Računarstvo
Profil: Računalno inženjerstvo
Mentor: izv. prof. dr. sc. Daniel Hofman

Zadatak: **Sustav za prenošenje 4K video snimke s niskom latencijom na male udaljenosti**

Opis zadatka:

Proučiti načine spajanja i prijenosa video signala s 4K kamere koja omogućava snimanje u sirovom obliku bez kompresije. Implementirati programsku podršku za prijenos videa s kamere na računalo. Osmisliti način prijenosa videa s računala do primateljske strane - putem bežične ili žične veze. Implementirati programsku podršku za prijenos podataka od pošiljatelja do primatelja. Implementirati dekodiranje video strujanja na primateljskoj strani. Testirati nekoliko algoritama za kodiranje videa pri prijenosu videa do primatelja. Usporediti kvalitetu i latenciju implementiranih načina prijenosa videa. Optimizirati prijenos videa i ponoviti testiranje i usporedbu. Napraviti dokumentaciju i objaviti izvorni kôd programskog rješenja na repozitoriju Git.

Rok za predaju rada: 23. lipnja 2023.

Sadržaj

Uvod	1
1 Organizacija i pristup radu	2
1.1 Zahtjevi sustava i izazovi njihovog određivanja	2
1.2 Pristup izrade rada	4
2 Arhitektura sustava	5
3 Zahtjevi i parametri izbora ulaznog člana	8
4 Pregled i usporedba dostupnih tehnologija.....	10
4.1 Općenito o sklopovskim i programskim sučeljima računala. Veza kamere i memorije enkodera	10
4.2 Video i podatkovna sučelja.....	13
4.2.1 HDMI.....	14
4.2.2 Display Port	14
4.2.3 SDI.....	14
4.2.4 Ethernet (NDI, GigE)	14
4.2.5 DCAM (FireWire - IEEE 1394)	15
4.2.6 Optika	15
4.2.7 USB	15
4.2.8 PCIe	15
4.2.9 Camera Link	16
4.2.10 CoaXPress	16
4.3 Upravljanje i napajanje.....	16
4.4 PCIe vs. USB latencija i propusnost. Host kontroleri	17
5 Izbor komponenata	18
5.1 Vrste kamera.....	18
5.2 Raw izlaz	19

5.3	Framerate i bitdepth.....	19
5.4	Senzor i zatvarač.....	20
5.5	Objektiv	20
5.6	Stabilizacija	21
6	Dostupne kamere visoke rezolucije	23
7	Dostupni Frame Grabber, odnosno Capture Card uređaji	24
8	Proučavanje konkretne kamere.....	26
9	Prvi prototip SPS-a	31
9.1	Tijek razvoja	31
9.2	Veza dva računala.....	31
9.2.1	Odabir formata i protokola za prijenos.....	32
9.3	Snimanje i prijenos pravog videa	34
9.3.1	Spajanje prave video kamere na računalo	34
9.3.2	Formiranje toka iz pravog videa. Odabir arhitekture, biblioteka i alata.....	34
9.3.3	Implementacija	36
9.3.4	Dekodiranje i prikaz	37
9.4	Tehničke značajke	37
9.4.1	Aplikacija.....	37
9.4.2	Kašnjenje komponenti i s kraja na kraj	40
9.5	Rezultati prvog prototipa SPS	43
9.6	Komentari	45
10	Drugi prototip SPS-a	47
10.1	Novi pristup.....	47
10.2	Poslužitelj i klijent	48
10.2.1	Konfiguracija rada	49
10.2.2	Čitanje i kompresija slika	49

10.2.3	Pakiranje slika.....	52
10.2.4	Prihvaćanje paketa.....	54
10.2.5	Obrada slika.....	55
10.3	Usporedba s prvim prototipom.....	60
10.4	Poteškoće mjerenja.....	63
Zaključak.....		70
Literatura.....		71
Sažetak.....		77
Summary.....		78

Uvod

Prijenos i gledanje video sadržaja u živo, što će reći bez dugotrajne pohrane i kasnijeg reproduciranja, danas je općeprihvaćena, utemeljena tehnologija. Kao takva, već je podložna sve strožim zahtjevima na svoja svojstva, primjerice glatkoći reprodukcije, kvaliteti slike i kašnjenju, ovisno o domeni gdje se koristi. Kod gledanja sportski događaja na televiziji, ne želimo da snimka zastajkuje, ima jasno vidljive artefakte ili kasni dovoljno dugo da čujemo reakciju susjeda na neki događaj prije nego se isti pojavi na našem ekranu. Današnjom tehnologijom to je moguće izvesti, no postoje i domene koje imaju drugačije i strože zahtjeve. Primjer takve, koja ujedno predstavlja potencijalnu primjenu ovog rada, je upravljanje bespilotnom letjelicom s udaljenosti od oko 100 metara, koristeći naočale za pilote dronova (engl. First Person View goggles, First Person View headset). U takvom scenariju, poželjna je velika prostorna rezolucija FPV naočala jer to osigurava jasnu sliku, i malo kašnjenje jer omogućava preciznije upravljanje, odnosno letenje bliže tlu i predmetima. Općenito govoreći, radi se o sustavu za prijenosu visoko rezolucijskog videa, reda veličine 4K, na niske udaljenosti s niskom latencijom, kraće SPS. U nadolazećim poglavljima bit će prikazana generička arhitektura sustava koji to omogućuje, proučene i uspoređene konkretne tehnologije za izvedu njegovih dijelova te demonstrirane dvije funkcionalne implementacije. Cilj rada nije realizirati proizvod koji može zadovoljiti konkretni skup zahtjeva, već utvrditi opću izvedivost, ključna ograničenja te okvirne performanse koje današnje opće namjensko sklopovlje može isporučiti.

1 Organizacija i pristup radu

1.1 Zahtjevi sustava i izazovi njihovog određivanja

Prvi korak kod razvoja novih proizvoda u pravilu je određivanje konačne funkcionalnosti iz koje proizlaze zahtjevi. Potom, ako nije sigurno da su zahtjevi ostvarivi, provodi se studija izvedivosti. Ako studija izvedivosti pokaže da prihvatljivu vjerojatnost uspjeha, slijedi razvoj samog proizvoda. Ovaj rad ne slijedi taj proces jer zahtjevi na SPS nisu fiksno definirani, a za to postoje dva razloga.

Prvi je kontradiktorna priroda funkcionalnosti te cijene i vremena razvoja: mogli bismo reći da želimo, primjerice, prijenos UHDTV videa, što će reći videa prostorne rezolucije 4 do 8K, s barem 60 sličica u sekundi (engl. FPS, Frames per second) te sirove podatkovne gustoće 144 Gbit/s, na udaljenost reda veličine 1 do 100 metara s manje od 100 milisekundi ukupnog kašnjenja. Ti zahtjevi na performanse nadilaze većinu danas komercijalno dostupnih tehnologija i proizvoda, posebice u kontekstu bežičnog prijenosa što znači da bi se vrlo vjerojatno bilo potrebno osvrnuti ka namjenskim rješenjima. Razvoj takvih je dugotrajan i skup te dovodi u pitanje je li tako nešto izvedivo u raspoloživom vremenu, jednoj osobi.

Drugi razlog proizlazi iz sljedećih pitanja: je li zaista potrebno da video bude 8K s 60 FPS? Što je od tih karakteristika bitnije: bismo li bili zadovoljni s 8K rezolucijom pri 30 FPS, ili bismo radije 4K, ali na 120 FPS? Kolika je minimalna potrebna kvaliteta slike i koliko smo spremni žrtvovati druge karakteristike da bismo ju postigli? Odgovor na ta pitanja možemo pokušati izvesti gledajući karakteristike ljudskog vizualnog sustava: FPV naočale koje bi dale doživljaj usporediv s realnošću imale bi vidno polje ne manje od 160° po horizontali i 150° po vertikali, uz oštrinu od 60 piksela po stupnju, što znači 9600 naspram 9000 piksela po oku te frekvenciju osvježavanja od 120 Hz [1]. Naš predloženi sustav ima 8K video, odnosno 7680 piksela na horizontali i 4320 na vertikali te 60 FPS, što je daleko manje od idealnih karakteristika i znači da se moramo pomiriti s kompromisima. Ako pak gledamo karakteristike prosječnih FPV naočala dostupnih na tržištu, vidimo da većina nudi kut gledanja manji od 50° i rezoluciju oko 1280×720 piksela, dok se na vrhu ponude nalaze naočale pune HD rezolucije, odnosno 1920×1080 s 100 Hz ekranom i oko 30 milisekundi

kašnjenja [2]. U usporedbi s time, predloženi zahtjevi na naš sustav osjetno su veći pa možemo zaključiti da bismo ih mogli i oslabiti bez većeg utjecaja na funkcionalnost.

Drugim riječima, u ovom trenutku nije moguće reći jesu li proizvoljno zadani zahtjevi ostvarivi u raspoloživom vremenu, niti je trivijalno jasno što bi poželjni zahtjevi uopće i bili. Zbog toga je odlučeno uzeti drugačiji pristup: ciljana rezolucija videa fiksirat će se na 4K, odnosno 3840 x 2160, utvrdit će se koji su izvori kašnjenja u prijenosu videa od kamere do monitora te će se pokušati minimizirati kašnjenje odabirom pogodnih i široko dostupnih sklopovskih i programskih komponenata. Razvijeno rješenje tada će biti svojevrsna studija izvedivosti, referentna točka za ostvarive karakteristike te podloga za daljnji razvoj, unaprjeđenje i optimizaciju. Dobit ćemo uvid u okvirne vrijednosti trajanje obrade UHD TV videa, stvarna kašnjenja pojedinih komponenti, konkretni obujam prometa i sl. te, ako se pokaže izvedivim, eventualno i pokušati podići rezoluciju.

Uzimajući u obzir gustoću podataka UHD TV video signala, faktore kompresije danas korištenih video koda i mogućnosti komercijalno dostupnih bežičnih tehnologije kratkog domet, korištenje bežične veze između dva računala komplicira izvođenje i predstavlja rizik jer bi moglo spriječiti puno ostvarenje ciljeva ako dostupna mrežna rješenja i tehnologije ne mogu prenijeti previđeni obujam prometa bez modifikacija ili nadogradnji. Stoga je prvi odabir jedne od komponenata napravljen odmah te je odlučeno koristiti žičnu LAN vezu, odnosno Ethernet s bakrenom paricom. Zahvaljujući slojevitoj arhitekturi OSI mrežnog modela, korištenje drugačijeg komunikacijskog medija ne utječe na više slojeve i stoga se smatra prihvatljivim. Od interesa za buduće verzije SPS-a koje koriste bežične tehnologije mogli bi biti WiFi 6 i 6E koji obećavaju brzine i do 9.6 Gbps, odnosno 19.2 Gbps, ali i rješenja poput 5G NR mmWave, 60 GHz wireless te Wireless Gigabit Alliance (WiGig). Njihove performanse su nešto slabije, no i dalje predstavljaju osjetno poboljšanje nad primjerice današnjim de facto standardom, WiFi 802.11ac s njegovih nominalnih bruto 1.3 Gbps propusnosti [52][53].

Ovdje se može napomenuti i da postoji mogućnost da je najbolje, odnosno minimalno kašnjenje kao posljedica rutinskih, kućanskih ili servisnih poslova samo po sebi dulje od razumnog perioda kašnjenja koje podrazumijeva pojam „niska latencija“ ili barem dovoljno dugo da nije moguće provesti kodiranje, prijenos i dekodiranje u preostalom vremenu. U tom slučaju bilo bi potrebno nabaviti napredniju komercijalnu odnosno izraditi namjensku opremu. Ako takva ne postoji ili je izvan budžeta projekta zaključiti da isti

trenutno nije izvediv u zadanom obliku ili prihvatiti nižu brzinu osvježavanja, lošiju kvalitetu slike ili sličan kompromis.

1.2 Pristup izrade rada

Prvi korak je modelirati cijeli SPS na apstraktnoj razini, od ulazne do izlazne komponente. Potom, proučavanjem postojećih rješenja, odgonetnuti koje konkretne opcije postoje za pojedine komponente, koji su njihovi ključni parametri, odnosno faktori usporedbe i rangiranja. Time će biti moguće odrediti optimalne karakteristike komponenata, ali i odrediti što je „dovoljno dobro“ za naš slučaj, a na čemu bi se u budućnosti moglo poraditi ako nismo zadovoljni s konačnim rezultatom.

Neke komponente, uglavnom sklopovske, su relativno fiksne po pitanju mogućnosti i zbog toga nude ograničeni potencijal optimizacije i unaprijeđena, isključujući mogućnost nabave boljih komponenti. Zbog toga je odlučeno više truda uložiti u razvoj i optimizaciju programskih komponenti, a kako bi to bilo lakše, inicijalno će se emulirati svi članovi SPS-a, a nakon njihove uspješne integracije i optimizacije, postepeno će se uvoditi stvarne komponente te u konačnici dobiti funkcionalan sustav.

Razvoj započinje ostvarivanjem toka podataka između dva računala preko IP stoga te spomenutog žičnog medija. Potom će se testni tok podataka zamijeniti programski generiranim video signalom kojeg će drugo računalo prihvatiti i prikazati na ekranu. Tada će biti moguće testirati razne formate video toka, algoritme i razine kompresije, strukture podataka i algoritme obrade signala itd. Konačno, slijedi spajanje prave kamere na ulaz koderskog računala i prijenos pravog toga na drugo računalo te optimizacija sustava u svrhu minimizacije kašnjenja prijenosa održavajući prihvatljivu kvalitetu slike.

Idealni ishod ovog procesa bio bi sklopovski prototip SPS-a, što obuhvaća video kameru, njen pogonski programi, sklopovlje za prikupljanje i slanje njenog izlaznog signala preko odabrane komunikacijske tehnologije, odnosno LAN mreže, te s druge strane obrnuto sklopovlje koje bi zaprimilo signal i pretvorilo ga nazad u format koji se može prikazati te prosljedilo na zaslon.

2 Arhitektura sustava

Konačni broj i raspored komponenata u sustavu nije jednoznačno definiran jer nisu ni zahtjevi, a također i jer odabir jedne komponente utječe na odabir druge, no počnimo od pretpostavke da je za sustav koji snima i prenosi video signal potrebna kamera, enkoder, mrežna oprema, dekodeer i zaslon. Zbog zahtjeva propusnosti i kašnjenja, koji će biti razmatrani u sljedećem poglavlju, može se pretpostaviti da će većina, ako ne i sve korištene komponente sustava biti diskretne, jer takve u pravilu nude bolje performanse, no to nije uvijek slučaj. Kamere poput FastVision FC1700 imaju integrirani FPGA čip za procesiranje slike. Ako bi koristili nju, spomenuti bi enkoder bolje bilo zvati poslužiteljem video signala jer zadaću kodiranja provodi sama kamera, no krenimo od onoga što je sigurno.

Ulaz u naš sustav predstavlja kamera. Ona mora zadovoljiti zahtjeve rezolucije te rada u stvarnom vremenu, a njenim odabirom određen je i vršni broj sličica u sekundi te početni format video signala. Zbog toga se nakon nje, po potrebi, nalazi sklopovlje za konverziju izlaznog signala u format koji sljedeći član u lancu može prihvatiti. Postoji više vrsta takvog sklopovlja, no zbog jednostavnosti ćemo koristiti općeniti naziv engl. Frame Grabber (FG), ili engl. Capture Card (CC). Eksterni CC ponekad nije ni potreban jer već dolazi integriran u kameri, ali kamera može zahtijevati druge dodatke: napajanje, upravljač za objektiv, dodatne vodove za upravljanje kamerom, rasvjetom i sl.

Osim nužnih komponenata, u fizičkoj izvedbi sustava mogu se javiti dodatni zahtjevi zbog koji bi u lanac biti potrebno dodati adaptere, konvertere i sl. Primjerice, ukoliko je jedan od zahtjeva mogućnost postavljanja kamere na udaljenosti od preko 100 metara od CC-a, a jedini medij koji može zadovoljiti traženu propusnost na toj udaljenosti je optičko vlakno, a kamera daje izlaz u elektroničkom formatu, tada bi sustav bilo potrebno proširiti medijskim konverterom.

Odabir CC-a zavisi i o sljedećem članu lanca, enkoderu, u čiju je memoriju potrebno učitati podatke koje izlaze iz CC-a, stoga već sada valja voditi računa o njemu. Enkoder, osim na izlazno sučelje CC-a ili medijskog konvertera, utječe i na protokol rada. Tu se primarno misli na razne DMA sheme za učitavanje podataka u grafički akcelerator, a njih diktira sklopovlje unutar enkodera. Glavne dvije su DGMA, implementirane u AMD

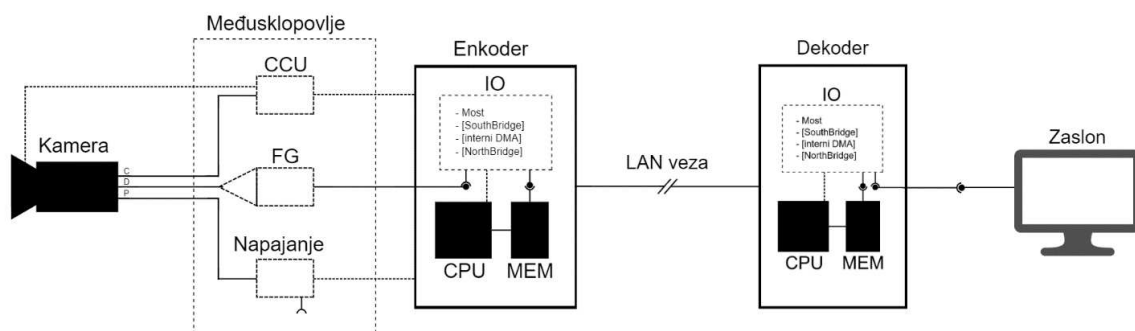
grafičkim karticama i GPUDirect RDMA, ekvivalent za Nvidia kartice, ali to može biti i vlastiti protokol ukoliko će biti korišten nekakav sklop po mjeri. Drugo važno svojstvo je UI sklopovlje enkodera koje može znatno utjecati na konačne performanse. Primjerice, ukoliko USB bude izabran kao konačni protokol, nije svejedno spajamo li enkoder na USB most koji je ekstenzija SouthBridge matične ploče, diskretni sklop spojen preko PCIe sučelja ili direktno integriran u CPU, no više o tome kasnije.

Spomenuti enkoder i njegov popratni dekoder imaju zadaću pakirati i slati, odnosno primiti i raspakirati ulazni video signal te, ako se za time pokaže potreba i ako to ne radi zasebni sklopovi, izvoditi sažimanje i obradu video signala. Za te svrhe postoji namjensko DSP sklopovlje, no za početak jednostavnije je koristiti osobna računala, pod uvjetom da posjeduju zadovoljavajuće sklopovske resurse.

Između enkodera i dekodera nalazi se žična LAN mreža, kao što je spomenuto u uvodu. Veza može biti ostvarena jednim križnim Ethernet kabel ili, jednostavnije, s dva Ethernet kabla i preklopnikom. Ovdje je najbitniji parametar propusnost, te se u svijetu Ethernet opreme koriste standardi brzine poput Gigabit Ethernet (GigE), 10 GigE i sl. kojeg sve komponente moraju zadovoljavati, od mrežne kartice do kablova i preklopnika.

Na izlazu sustava nalazi se zaslon, spojen na dekoder analognim ili digitalnim sučeljem poput HDMI, DisplayPort, DVI, SDI i sl. I tu se uvodi kašnjenje u sustav jer zaslon ima ulazno kašnjenje, latenciju obrade signala te vrijeme odaziva piksela, pa se ti faktori trebaju uzeti u obzir tijekom analize kašnjenja.

Sve navedene komponente zajedno su prikazane na slici 1. Crtkane komponente i veze označavaju opcionalne, odnosno uvjetne komponente. Primjerice, napajanje kamere može biti izvedeno kao zasebni sklop napajan iz gradske mreže, a može biti i povezano direktno na ostatak sustava (enkoder). Ili, uopće ne mora biti prisutno jer se kamera napaja preko iste veze kojom teku podaci i upravljački signali. Upravljačka, podatkovna i linija napajanja označene su redom slovima 'C', 'D' i 'P'.



Slika 1 Opća arhitektura sustava

3 Zahtjevi i parametri izbora ulaznog člana

Poznati zahtjevi su rezolucija, 4K, te minimiziranje latencije. Znajući rezoluciju, broj i dubinu kanala boje te broj sličica u sekundi, možemo grubo izraziti potrebnu propusnost sustava prema izrazu (1):

$$T = X * Y * n * c * p \left[\frac{b}{s} \right] \quad (1)$$

Gdje su X i Y horizontalni i vertikalni broj piksela slike, n broj bitova po kanalu, c broj kanala te p broj sličica u sekundi. Okvirno, za 4K rezoluciju, poznatu i kao UHD, koja ima 3840×2160 piksela, tri 8-bitna kanala boje (R, G i B) te 60 FPS, dobivena brzina iznosi 11,124 Gbit/s, odnosno 1,390 GB/s. Za više rezolucije (8192x4320, 8K Full Format), veću dubinu boja (10-bit, 12-bit i sl.) taj broj može biti i znatno veći, ali mi ćemo se ograničiti na ove vrijednosti. Također, kamere često izvide neki oblik kompresije bez (vizualnih) gubitaka, čak i ako proizvode sirovu/neobrađenu snimku (engl. RAW). Količina kompresije je u pravilu podesiva te se izražava kao omjer teoretske i stvarne brzine izlaza, npr. 1:3, 1:7 i sl. U svakom slučaju, sklopovlje iza kamere, uključujući i komunikacijska sučelja i kablove, mora moći podnijeti te brzine i pritom unositi što manje kašnjenja [3]. Zato se gotovo u potpunosti fokusiramo na kamere koji podržavaju spomenuti RAW izlaz te nude pogodna komunikacijska sučelja. Više o tome u sljedećem poglavlju [4].

S obzirom na to da nije navedeno drugačije, pretpostavit ćemo da je udaljenost kamere, međusklopovlja i enkodera ispod 10 metara. To je potencijalno bitan faktor jer diktira upotrebljiva sučelja, odnosno kablove. Na primjer, bude li korištena kamera s HDMI izlaznim sučeljem, maksimalna udaljenost odgovara ograničenju duljine HDMI kabla, odnosno oko 15 metara [5]. Ukoliko želimo li smanjiti latenciju korištenjem kamere koja ne treba CC, nego direktno nudi npr. USB 3.2 Gen1, maksimalna duljina pada na 2 metra. Kod USB 3.2 Gen2 ograničenje je 1 metar, a, ovisno o izvedbi, duljina PCIe kablova varira od nekoliko metara pa do samo par desetaka centimetara [6][7].

Pod pretpostavkom da će postavke kamere i objektiva trebati mijenjati (zum, fokus, blenda, zatvarač, osjetljivost ...) i s obzirom na to da su sustav za snimanje i prikaz slike prostorno odvojeni, moglo bi biti poželjno imati udaljeno upravljanje kamerom i/ili

objektivom. Implementacije ovoga su raznovrsne, od integriranih sustava gdje driver instaliran na računalo preko žične (USB) ili bežične (WiFi, Bluetooth) veze od kamere prima informacije te povratno šalje naredbe, a kamera objektivu proslijedi njegov dio, primjerice preko SPI protokola kod EF nosača objektiva (engl. lens mount) [8]. U drugim sustavima se pak kamerom i objektivom upravlja zasebno, uz pomoć dodatnih vanjskih upravljača koji stoje između operatera i kamere [9]. Ovakav pristup čest je na kamerama koje ne podržavaju EF ili slične objektivne. Još su složeniji sustavi korišteni na filmskim setovima, gdje opremom upravljaju posebni motorizirani sklopovi koji se montiraju na objektiv ili kameru, te su uz njih prisutni specijalizirani studijski kontroleri, mikseri i slično [10]. Zbog te kompleksnosti u ovom trenutku ne možemo donijeti čvrstu odluku koji će se pristup odabrati, no valja imati na umu da ukoliko odabrana kamera nazivno i ne podržava udaljeno upravljanje, ono je i dalje često izvedivo, no na uštrb jednostavnosti.

Valjda se dotaknuti i mogućnosti kamere i objektiva, odnosno na kvalitetu slike koju oni proizvode. Kakve se performanse očekuju u uvjetima slabog osvjetljenja? Što je subjekt snimanja? Koliko je brz i udaljen od kamere? Je li kamera statična? Gdje se odvija snimanje? Je li kamera izložena okolišu koji nadilazi njene mogućnosti? Odgovori na ta pitanja postavljaju ograničenja ili uvijete na veličinu senzora, način očitavanja slike (global, rolling), toleriranu količinu šuma, na veličinu otvora blende, kut i dubina gledanja (engl. angle of view, focal length, depth of field), brzinu zatvarača, robusnost (otpornost na šok, vibracije, vlagu, špricanje, toplinu, ...) itd. Ukoliko se kamera treba micati, poželjna je ugrađena, eksterna sklopovska ili eventualno programska stabilizacija slike te tada težina i dimenzije ulaznog člana također ulaze u razmatranje. Konačno, ako sustav nema garantiran pristup gradskoj mreži ili drugom neometanom izvoru napajanja, potrošnja te kapacitet baterija postaju bitni.

4 Pregled i usporedba dostupnih tehnologija

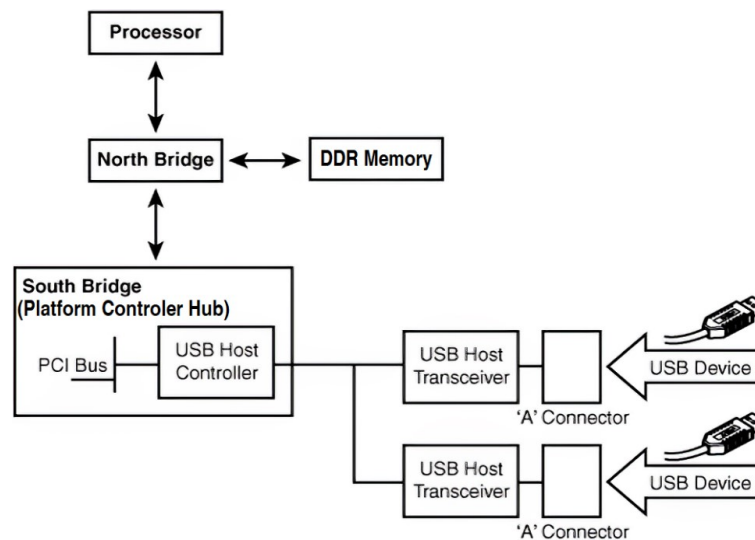
Zahtjevi našeg sustava relativno su nestandardni. Većina video sadržaja i dalje je u rangu 1920x1080 (Full HD, 1080p) jer prednosti veće rezolucije uglavnom ne opravdavaju potrebnu količinu računalne snage, prostora ili propusnost internetske veze. Ako se usredotočimo na svijet živog prijenosa, osim Olimpijskih igara u Tokyu, rijetko tko nudi 8K sadržaj, dok su 2 i 4K izvori nešto češći, ali u pravilu bez zahtjeva da kašnjenje bude manje od jedne sekunde, a kamoli 100 ms. Zbog toga trenutno ne postoji pregršt zrelih tehnoloških rješenja koja bi omogućila tako nešto pa, kako bismo znali kakva to oprema može zadovoljiti naše potrebe, u sljedećih par poglavlja bit će prikazano što je trenutno dostupno.

4.1 Općenito o sklopovskim i programskim sučeljima računala. Veza kamere i memorije enkodera

U svrhu boljeg razumijevanja konkretnih sučelja koja slijede, ovdje je dan opis uobičajenih sklopovskih i programskih komponenata koje stoje između ulaznih uređaja i radne memorije modernih računala. S obzirom na to da moderna računala nemaju mogućnost „čitanja“ video sadržaja direktno s video sučelja, u tu se svrhu koriste proširenja: aktivno ili pasivno sklopovlje koje konvertira izlazno video sučelje kamere u neko od ulaznih sučelja koja matična ploča nudi. Oni se nazivaju engl. Frame Grabber (FG) ili Capture Card (CC). Pojam FG dolazi iz analognog svijeta, odnosno opisuje uređaj koji je analogni video signal digitalizirao, odredio eventualnu naknadnu obradu poput smanjenja šuma te dobiveni digitalni signal poslao dalje, najčešće prema trajnoj memoriji ili računalu. Danas, s obzirom na to da većina kamera daje digitalni izlaz na svojim izlazima može eventualno postojati potreba za prilagodbu formata, konverziju sučelja ili slično. Prigodniji naziv za uređaje koje obavljaju takve zadatke je CC, no kolokvijalno se koriste oba termina za opis potonjeg.

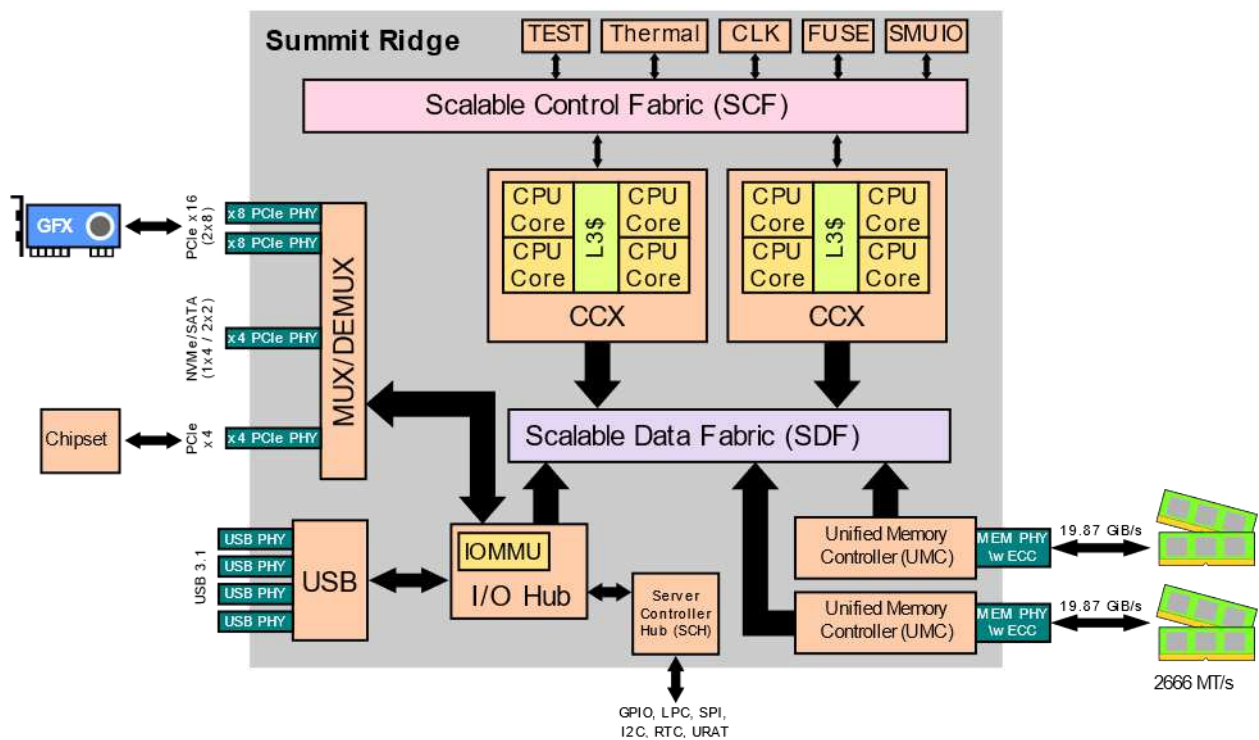
CC se s memorijom ili procesorom najčešće povezuje (A) preko USB sučelja koje vodi do PCIe sabirnice ili (B) direktno na PCIe sabirnicu. Ukoliko se koristi USB sučelje, put podataka vodi od USB uređaja do USB Host upravljača, koji može biti diskretan ili

integriran u Chipset/Southbridge matične ploče (nekada poznat i kao Platform Controller Hub) [9]. Od tamo se vodi do memorijskog upravljača (NorthBridge, danas u pravilu integriran u CPU) preko DMI ili UMI sabirnice. Takve su arhitekture prikazane na slikama 4 i 5 [14] [15]. Na starijim sustavima za tu vezu koristio se PCI. Tek sada podaci mogu biti pospremljeni u RAM i poslani na obradu u CPU, GPU ili negdje drugdje. Skica ove arhitekture prikazana je na slici 2 [12][13].

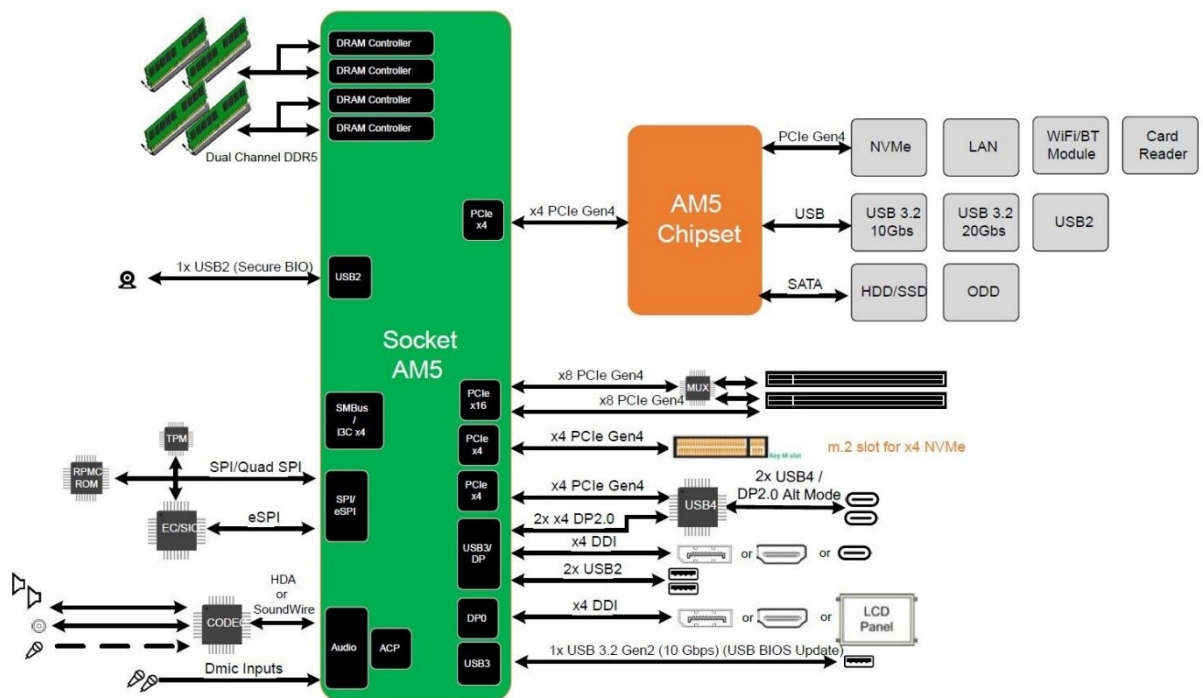


Slika 2 Put podataka USB-a

Sada je jasno i zašto CC koje mogu direktno koristiti PCIe u pravilu imaju manje latencije: na putu im stoji manje komponenata, formata i konverzija [16]. Treba imati na umu da su CPU integrirani USB Host kontroleri relativno nova inovacija te da većina arhitektura još ne nudi takvu opciju. Također, teoretska ograničenja propusnosti za 4x ili 8x PCIe Chipset Uplink (Intel DMI 3.0 i 4.0), sam Chipset/PCH te za diskretni USB Host kontroler treba uzeti sa znom soli, jer konačne performanse uvelike ovise o kvaliteti implementacije. Nadalje, eksterni USB Host kontroleri u pravilu koriste PCIe sučelje, no mogu biti spojeni na Chipset/PCH PCIe ili, od relativno nedavno, direktno na CPU/Northbridge PCIe linije (engl. lane), ako one nisu ograničene na M.2 SSD. Ovi su koncepti ilustrirani na slikama 3 i 4.



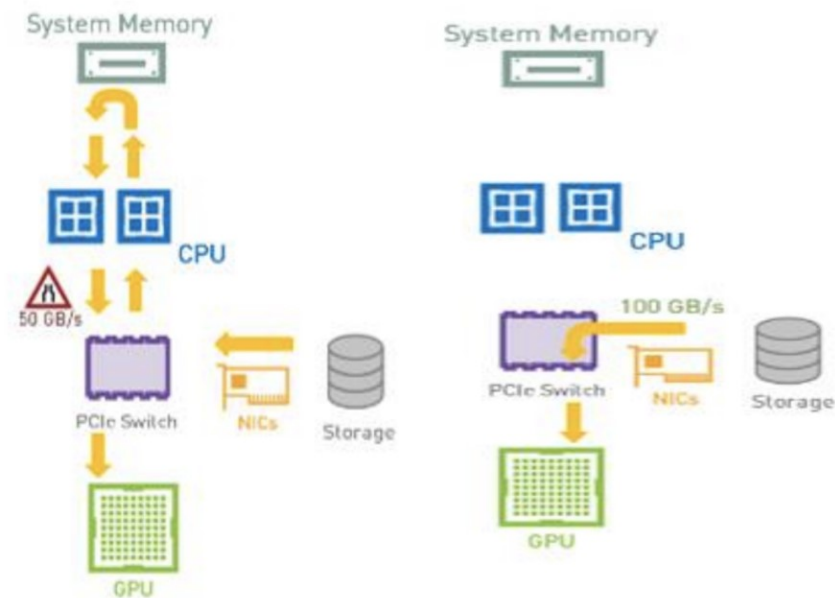
Slika 3 Prikaz AMD Summit Ridge CPU jezgre



Slika 4 Zamišljena AM5 matična ploča

Ovdje ulogu igra i driver stog, čija kvaliteta u konačnici odlučuje iskorištavaju li se sklopovski resursi optimalno ili ne. To uključuje drivere za „host controller“, „hub driver“ i „USB device client“ [17][18].

Ovakva se organizacija može dodatno unaprijediti i to na dva načina: gore spomenutom direktnom ugradnjom USB Host kontrolera u CPU ili korištenjem direktne veze CC i radne memorije GPU-a, ako se GPU koristi za obradu video signala. Nvidia potonju organizaciju naziva GPUDirect Storage i njena gruba organizacija prikazana je na slici 5. Kompanija tvrdi na originalnih 50 GB/s protoka može poboljšati na više od 200 GB/s [19][20].



Slika 5 Nvidia GPUDirect Storage put podataka

U kombinaciji s bibliotekama za obradu slike u GPU, poput besplatnog NPP (NVIDIA Performance Primitives), NVIDIA Video Codec SDK i GPUDirect for Video, Windows AVStream drivera ili komercijalnih rješenja poput Fastvideo SDK, video je moguće zaprimiti, obraditi i pohraniti ili prikazati isključivo koristeći GPU. Fastvideo tvrdi da, koristeći XIMEA xiB-20 (5120x3840, 12-bit, 30 fps) s PCIe izlaznim sučeljem i driverima koji podržavaju GPUDirect RDMA te Nvidia GeForce GTX Titan, ukupno kašnjenje učitavanja, obrade i pohrane iznosi svega 13.7 ms [21]!

4.2 Video i podatkovna sučelja

Cilj ovog poglavlja dati je krati uvid u video i podatkovna sučelja danas korištena, među ostalim, za video prijenos.

4.2.1 HDMI

Jedan od najrasprostranjenijih, nekomprimiranih, digitalnim video standarda, HDMI definira sam protokol i signal, ali i sučelje i mehaničke standarde sučelja. Posljednji, 2.1 standard, omogućuje 48 Gb/s propusnost što je dovoljno za RAW 8k@60p s 8bit RGB ili YCBCR 4:4:4 video te teoretski do 20m, no u praksi do 8m udaljenost. Podržava i 3:1 DSC kompresiju što proširuje iskoristive video formate. Pravo implementacije i prodaje naplaćuje se godišnje i po prodanom komadu.

4.2.2 Display Port

Glavna konkurencija HDMI-u, s kojim dijeli mnogo karakteristika. Zadnja verzija, 2.0, pruža 80 Gb/s na garantirano 3m, no u praksi i više, DSC 1.2 kompresiju te prijenos drugih paketnih protokola poput USB-a i Ethernet preko dodatnog 8b/10b 576 Mb/s kanala. Iako vlasnik standarda, VESA, ne naplaćuje pravo na implementaciju ili prodaju uređaja s DP konektorom, pristup standardu dobiva se samo uz VESA članstvo.

4.2.3 SDI

Serial Digital Interface standardiziran od SMPTE obuhvaća više sučelja (ITU-R BT.656, SMPTE 259M, 292M, ...). Zadnje među njima je 12G-SDI s 12 Gb/s na 100m, no nema ograničenja na broj kablova, stoga se danas može vidjeti u 4x konfiguraciji u 8k60p sistemima. Koristi praktične MINI i BNC SDI konektore, no nešto je manje praktična politika licenciranja kada se prenosi neenkriptirani sadržaj, zbog čega se rijetko vidi na potrošačkoj elektronici gdje ga je uvelike pretekao HDMI.

4.2.4 Ethernet (NDI, GigE)

Video prijenos preko Ethernet sklopovlja i protokola. Glavna prednost je rasprostranjenost opreme te relativno mala cijena TPC kablova te velika propusnost i udaljenosti. Primjerice, Cat6 standard koji, za razliku od novijeg Cat7, i dalje koristi RJ-45 (8P8C) konektor te garantira 10 Gb/s protoka na 100m duljine. Kod video prijenosa koriste se GVCP i GVSP protokoli za upravljanje odnosno prijenos podataka. Na žalost, RT karakteristike uvelike ovise o stabilnosti temeljne LAN izvedbe, stoga ne postoje stroge garancije za pouzdanost, kapacitet, kašnjenja i sl. Uzme li se u obzir mogućnost autodetekcije uređaja te presretanja

GVSP paketa u IP stogu prije nego stignu opteretiti CPU, ovo je sučelje pogodnije za uporabe poput video nadzora, no upotreba za RT svrhe već potvrđena u praksi [22].

4.2.5 DCAM (FireWire - IEEE 1394)

Stari standard, ograničen protokom (400 MB/s) te sa sve manje i manje dostupnog sklopovlja koje ga nudi, odnosno podržava. Ako ništa drugo, mali protok ga čini neupotrebljivim za naše potrebe.

4.2.6 Optika

Navedeno kao sučelje zbog ARIB, EBU i SMPTE fibre standarda (IEEE ST 304:2009), odnosno SMPTE 311M Fibre kabla te SMPTE 304 Lemo konektora (3K.93C). To je kompozitni, odnosno hibridni standard koji sadrži optiku i bakar te u jednom kompaktnom i čvrstom (npr. PUR standard) pakiranju prenosi napajanje, upravljački signal, zvuk i video. Iako sam po sebi ne definira sučelje, koristi se kao fizički sloj za ostvarivanje istih na većim udaljenostima i kapacitetima, uglavnom preko frekvencijskog multipleksa (npr. Multi 12G-SDI over SMTPE Fibre, tzv. U-SDI). Na taj način se već korištene konfiguracije, poput 4x 12G-SDI sučelja za prijenos 8K signala, uz zasebno napajanje i upravljanje mogu izvesti znatno elegantnije [23].

4.2.7 USB

Gledajući specificiranih 20 Gb/s uz duljinu kabla do 1m te 4.5W na 5V, u kontekstu drugih ovdje opisanih sučelja lako ga se može opisati, no uzimajući u obzir njegovu rasprostranjenost, direktnu podršku Intel i AMD chipseta (DMA prijenos u RAM bez uključenja CPU-a), GenICam podršku, latenciju reda veličine 10 do 100 mikrosekundi, zahvaljujući uklanjanju polling sistema, i on može biti prihvatljiv [24].

4.2.8 PCIe

Serijski standard za računalne sabirnice, korišten za prijenos slike te podataka (HDD, SSD i internet). Također služi kao temeljni standard za ExpressCardm SATA Express, U.2 i M.2, preko kojeg se može izvesti i USB 3. Sastavljen od između 1 i 16 paralelnih linija, najnovija 6.0 verzija podržava do 121 GB/s, iako većina uređaja još nudi 5.0 verziju (3.9 GB/s po liniji, 63 GB/s za 16x). Podržava napajanje do 9.9W na 3.3V. Zbog toga što je on

de facto standard za internu PC komunikaciju, procesori i matične ploče u većini slučajeva nude svoje vršne performanse baš preko PCIe sabirnice, odnosno drugi komunikacijski protokoli izvede se povrh njega. Zbog toga, periferija koja se direktno spaja na PCIe ima direktni pristup CPU, GPU, RAM, DMA i drugim interesantnim uređajima.

4.2.9 Camera Link

Sučelje dizajnirano za kamere iz nule, a propisuje i standard za dodatnu opremu poput CC-a. Ima „power over interface“ mogućnost te upravljanje integrirano na zasebnom serijskom kanalu uz osnovni podatkovni kanal. Bez pojačala ili druge opreme ograničen na 10m, no u „punoj konfiguraciji“ (engl. Deca, Extended Full) koristeći 2 kabla, nudi 6.8 Gb/s kapacitet što bi moglo biti nedostavno, no postoji i HS inačica (CLHS) koja nudi 16.8 Gb/s, RT mogućnosti te garanciju ispravnosti prijenosa preko FPGA implementiranog fizičkog sloja kako bi se osiguralo malo kašnjenje. HS specificira i kablove do 10 km dometa, ukoliko je tako nešto potrebno. Inačice izvedene optikom mogu ići i do 80 GB/s [25].

4.2.10 CoaXPress

U misiji usporediv s CL sučeljem, CoaXPress inačica 2.0 nudi 12.5 Gb/s do 30m po kablju, no standard ne ograničava broj kablova stoga se može naći oprema koja koristi 8 paralelnih veza čime se postiže 4.8 GB/s. Podržan je i „power over interface“, no do 13W na 24V. Također poput CL-a, koristi protokol baziran na paketima s očuvanjem integriteta podataka, brzo „uplink“ upravljanje, GPIO te okidanje, sve preko jednog kabla. Treba napomenuti i prednost BNC konektora korištenih s RG6/11/59 kablom: zaključavanje, odnosno prevencija ispadanja što je korisno u mobilnim sistemima.

4.3 Upravljanje i napajanje

Upravljanje i napajanje kamera može se izvesti na velik broj načina, no neki od njih prevladavaju u određenim kontekstima. Potrošača elektronika danas često koristi bežična sučelja za upravljanje (Wi-Fi, Bluetooth) i primarno napajanje baterijama ili USB kablom. Profesionalna i polu profesionalna oprema koristi RF upravljače, RS-485 ili 422 izveden kao DMX512, SDI, SPI ili sličan protokol preko XLR, Hirose, BNC, M8/12, Lemo E/F/M konektora (i upravljanje i napajanje) [26][27][28]. Manji broj uređaja oslanja se na upravljanje preko sporednih kanala HDMI i DP sučelja, IP protokol (Ethernet) ili

specijalizirane veze, poput SMPTE 311M. Konačno, industrijske kamere znaju koristiti nestandardizirana GPIO sučelja, poput XIMEA PCIe kamera.

4.4 PCIe te USB latencija i propusnost. Host kontroleri

Interni, odnosno integrirani u SB, i eksterni sklopovi koji omogućuju USB komunikaciju nazivaju se Host Controller (HC). Oni implementiraju engl. „host controller interface“ (HCI), odnosno omogućuju komunikaciju USB uređaja s driverima u OS-u (engl. Host Controller Driver, HCD). Preko njih ide i prijenos podataka („Pipes“) i upravljanje USB uređajima, uključujući „handshake“, konfiguraciju brzine i napajanja, prekide (npr. „wakeup“) i sl. S obzirom na to da je HCI standardiziran, HC i HCI se mogu raditi nezavisno, odnosno OS može imati generički driver koji radi sa svakim HC, iako specijalizirani driveri (ASMedia, RME, ...) mogu poboljšati performanse.

Moderniji protokoli (xHCI) daleko unaprjeđuju performanse, odnosno bolje barataju novijim USB protokolima (USB 3.1 gen 1/2, 3.2 gen 1/2/2x2). Primjerice, omogućuju I/O akcije uređaja bez čekanja na obradu prekida (engl. polling), odnosno mogu slati podatke asinkrono od Hosta (i CPU-a) te imaju zasebne data i command linije. Usprkos tome, zbog block transfera, buffera i sl. u pravilu i dalje imaju veće latencije od PCIe, pogotovo ako se koristi DMA, i to do nekoliko redova veličine [34][35]! Modernije inačice protokola ne pate od problema sinkronih USB transakcija u istoj mjeri kao stariji, no i dalje ih treba imati na umu kada razgovaramo o performansama.

5 Izbor komponenata

Osnovni zahtjev na kameru je izlazna 4K (UHD) ili usporediva rezolucija, no ovisno o području primijene, postoji niz drugih atributa kamere koji ograničavaju naše opcije ili služe kao sekundarni kriteriji odabira. U ovom poglavlju u kratko će biti navedeni neki od njih, a potom dan popis danas dostupnih te ili veće rezolucije i njihovih karakteristika. Nakon poglavlja 4 upoznati smo bitnim izlaznim i upravljačkim sučeljima kamera te kako ona komuniciraju s procesorom enkodera, stoga ovdje taj dio neće biti dalje diskutiran.

5.1 Vrste kamera

Podjela kamera uglavnom se radi po ciljnoj namjeni i formatu, no danas, zbog razvoja DSLR kamera, jasne crte između kategorija postaju sve mutnije. Ipak, kamere se u pravilu dijele na amaterske, kompaktne te profesionalne, odnosno film ili TV kamere. Potonje su najčešće izvedene kao Point-and-shoot, Single-lens reflex (SLR) ili Mirrorless, gdje ti termini opisuju strukturu mehanizma za pregled scene prije i/ili tijekom snimanja, tzv. tražilo (engl. Viewfinder). PAS kamere imaju diskretno tražilo, čija perspektiva ne odgovara u potpunosti perspektivi senzora, stoga snimljena slika nešto odudara od onoga što operater vidi. SLR i Mirrorless rješavaju taj problem: SLR koristi sistem špiгла i prizmi kojim, dok kamera ne snima, preusmjerava perspektivu senzora u tražilo, dok Mirrorless izravno prikazuju sliku iz senzora na tražilu, što znači da kamera konstantno snima i dekodira, samo je pitanje pohranjuje li se sadržaj u memoriju ili ne.

Profesionalne kamere, bilo za film ili TV, u pravilu su veće od amaterskih te dolaze bez sve potrebe opreme – ista se odabire i nabavlja specifično za konkretnu uporabu. Prednost tog pristupa je znatno veća fleksibilnost sustava, no to dolazi s višom cijenom, veličinom, kompleksnošću i sl. U pravilu se, osim osnovnih funkcionalnosti kamere poput zuma i fokusa, parametri podešavaju unaprijed, a samo po potrebi korigiraju tijekom rada, često iz udaljenog upravljačkog centra. S obzirom na to da je jedini zadani strogi uvjet na kameru 8K rezolucija, deklarirana svrha i format kamere nisu nam ključni faktori odabira.

5.2 Raw izlaz

Potrošačke kamere, kako bi uštedjele na memorijskom prostoru i olakšale rad korisnicima, u pravilu ne spremaju originalni signal iz sklopa za obradu. Naime, neovisno koristi li se CMOS, CCD ili neka druga vrsta senzora u kameri, izvorni izlazni signal tih komponenti u pravilu zahtjeva dodatnu obradu. Ovdje se ne misli na umjetničku, naknadnu obradu u nekom od računalnih alata, poput Photoshopa, Corela i sl., nego funkcionalne alteracije signala. Tu operaciju izvodi tzv. Image Procesor (IPU, ISP), a sastoji se od mnogo pod operacija, uključujući transformaciju iz Bayer formata u RGB, YUV, YIQ i slične formate boja, ublažavanje šuma, izoštravanje, korekcija svjetline i game, potiskivanje posljedica micanja kamere pomoću podataka iz ugrađenog akcelerometra (ukoliko isto nije odrađeno sklopovski), prilagođavanje rezolucije itd. Konačno, obrađena slika se komprimira, često u jpeg format i pohranjuje. No, kamere predviđene za rad u znanosti, industriji i sličnim zahtjevnim okolinama te obrade prepuštaju vanjskom sklopovlju te na izlazu daju neobrađeni (engl. RAW) format. Ova mogućnost daje priliku za unaprjeđenje latencije sustava jer vanjski IPU može biti puno moćniji od integriranog, stoga pri odabiru dajemo prednost kamerama koje ju nude.

5.3 Framerate i bitdepth

Broj sličica u sekundi (engl. Framerate, FPS) i raspon vrijednosti piksela (engl. Bit depth, Color depth) koje je kamera u mogućnosti isporučiti pri zadanoj rezoluciji varira od uređaja do uređaja. Većina monitora, televizora i projektoru dostupna danas ograničena je na 8-bit raspon vrijednosti, a maksimalni efektivni framerate koji mogu prikazati diktira brzina osvježavanja slike (engl. Refresh rate), obično izražena u Hercima. S obzirom na to da ne prikazujemo usporenu ili ubrzanu snimku (engl. Timelapse), brzina prikaza odgovara broju sličica u sekundi izvornog video zapisa, odnosno brzini osvježavanja monitora, što je danas obično 30 ili 60, no može biti i 75, 120 ili rjeđe 144 do 240 Hz. Kao okvirnu vodilju, reći ćemo da kamere koje pri 4K rezoluciji mogu pružiti 30 do 60 FPS pri 8-bit rasponu vrijednosti piksela zadovoljavaju naše potrebe, no veći FPS znači i brža reakcija na vanjsku pobudu pa prednost dajemo bržim kamerama.

5.4 Senzor i zatvarač

U poglavljima 3 i 5.2 već su spomenuti senzori i njihova svojstva koja diktiraju odabir kamere, stoga će ovdje biti dan samo kratki opis. Danas je većina senzora izvedena u CMOS tehnologiji, no CCD se i dalje može pronaći u zahtjevnijim aplikacijama. To je zato što CCD u pravilu daje oštrije slike s manje šuma od CMOS te je osjetljiviji na svjetlo pa ima prednost u tamnijim okruženjima, iako se razvojem CMOS razlike smanjuju. S druge strane, CMOS je jeftiniji, troši manje energije i, bitnije, zbog paralelne prirode skeniranja piksela, omogućava brže iščitavanje vrijednosti pri usporedivoj količini šuma nego CCD. Zbog toga CMOS vidi puno više primjene u područjima poput strojnog vida, zbog čega je i bolji izbor za nas.

Sekundarni parametri uključuju fizičku veličinu senzora, čije povećanje poboljšava svjetlosnu osjetljivost odnosno umanjuje šum jer je potrebna manja količina pojačanja signala za postizanje iste svjetline. Tu je i metoda očitavanja piksela (ako se koristi elektronički zatvarač), uglavnom engl. Global/Total ili Rolling, što znači istovremeno zapisivanje cijelog senzora odnosno zapisivanje jedne po jedne horizontalne linije. Rolling zatvarači su uglavnom jeftiniji, no slika koju proizvode ne odgovara stvarnosti zbog distorzije (engl. skew, wobble), koja je izraženija kod brzih subjekata. Global s druge strane proizvodi vjerniju sliku, no zahtjeva naprednije ADP sklopovlje pa i košta više. Konačno, treba spomenuti i fizički zatvarač. On može biti izveden u kameri ili objektivu te je u pravilu sporiji od elektroničkih, unutar i između okidanja, te proizvode vibracije i artefakte poput odblijeska. Elektronički zatvarači zaostaju iza mehaničkih samo kod dugih ekspozicija i sinkronizacije s eksternim bljeskalicama. Zaključno, optimalan izbor za nas je CMOS senzor s hibridnim ili čistim elektroničkim zatvaračem.

5.5 Objektiv

Konkretni parametri odabira objektiva direktno proizlaze iz korištene kamere. Ona određuje dostupni nosač objektiva (ako se ne koristi proizvoljni adapter), poziciju ravnine senzora koja definira potreban kut pokrića (engl. angle of coverage) i eventualnu nisku osjetljivost, za što je potrebno objektivnom osigurati više nego prosječnu količinu svjetla. Dalje, rezolucija kamere diktira potrebnu kvalitetu objektiva, odnosno oštrinu slike koji on mora moći pružiti. Drugi definirajući faktor je udaljenost i dimenzije subjekta te jesu li oni statički ili pokretni. Objektivi koji podržavaju promjenu žarišne duljine, a time i kut

gledanja, odnosno vidno polje (engl. Angle of View, Field of View) nazivaju se zum objektivima. Podskupina njih su parafokalni (engl. true zoom, parafocal) objektivima koji mehanički održavaju fokus prilikom promjene žarišne duljine. „Cijena“ ove mogućnosti je povećana težina, dimenzije i cijena, sporiji automatski fokus te potencijali gubitak na kvaliteti slike, pogotovo pri rubovima kadra kada se približavamo rubnim žarišnim duljinama na objektivima s velikim rasponom. U kontrastu njima, objektivima koji imaju fiksnu žarišnu duljinu nazivaju se primarnim objektivima (engl. prime lens, Fixed Focal Length). Oni nemaju probleme podesivih objektivima, no zahtijevaju fizičku izmjenu objektivima ukoliko se želi promijeniti žarišna duljina.

Neovisno o kameri, još jedan važan parametar objektivima je otvor blende i njemu povezana dubina polja. Pored duljine ekspozicije, izražene kao dio sekunde, i osjetljivosti senzora zvane ISO, otvor blende (engl. f-stop, f-number) regulira količinu svjetla koja dopire do senzora. Računa se kao inverz omjera otvora objektivima (engl. aperture) i žarišne duljine, npr. $f/1.4$, $f/2$, $f/2.8$ itd., gdje svaki sljedeći korak prepolovljuje efektivnu površinu svjetlosnog otvora. Veći otvori blende omogućuje da više svjetla dođe do senzora, no paralelno umanjuje dubinu polja, odnosno zamišljeni prsten u vidom polju kamere u kojem su subjekti u fokusu. Obratno, uz konstantan fokus, smanjenjem otvora blende će veći raspon subjekata, gledano naspram udaljenosti od kamere, biti oštar. Teoretski maksimalan otvor koji neće proizvoditi sferne aberacije je $f/0.5$, no danas u pravilu objektivima ne idu ispod $f/1.2$ ili $f/1.4$, uz par iznimki, poput Nikonovog Noct 58mm $f/0.95$ objektivima, koji košta pozamašnih USD 8000 [29].

Za naše svrhe, primarni ili zum objektiv s mogućnosti motoriziranog upravljanja, poput onih s EF nosačem, s maksimalnim otvorom $f/2.8$ bio bi zadovoljavajući. Ukoliko se zahtjevi snimanja postrože uvijek postoji opcija nabavke profesionalnih TV objektivima [30].

5.6 Stabilizacija

Ukoliko se kamera mora pomicati tijekom snimanja poželjan je neki oblik stabilizacije slike. Moderne DSLR kamere često imaju sklopovsku stabilizaciju ugrađeno u samo tijelo kamere (engl. IBIS) ili u objektivu (engl. OIS, IS, OS) i to na dvije (X i Y) do pet osi (X, Y i sve 3 rotacijske), dok filmske i TV kamere u pravilu koriste eksternu stabilizaciju preko kardana (engl. gimble), kolica (engl. dolly), ruku ili složenijih mehaničkih sklopova [31][32].

Postoji i treća opcija: programska stabilizacija odrađena pri obradi signala, no zbog računalne složenosti ovog postupka njega primjena u RT okruženju nije idealna [33]. S obzirom na to da još nije poznato koja kamera će se koristiti, niti hoće li biti pokretna ili nepokretna, u ovom trenutku nije moguće reći je li stabilizacija nužna i, ako je, u kojem obliku.

6 Dostupne kamere visoke rezolucije

U prilogu uz rad nalazi se tablični popis visoko rezolucijskih kamera koji je danas dostupan za nabavi. Uz svaku kameru u tablici su navedene i karakteristike koju su diskutirane u 5. poglavlju. Jedini bitan faktor koji nije uključen je cijena, jer dobar dio proizvođača i dobavljača tu informaciju ne objavljuje javno, nego na upit. Uz to, kamere često postoje u više inačica te budu upakirane s nekom dodatnom opremom pa cijena dodatno varira.

7 Dostupni Capture Card uređaji

Prednosti kamera koje se mogu spojiti na računalo direktno, po mogućnosti preko PCIe ili USB sučelja diskutirane su poglavljima 4. i 7.1. No, postoji mogućnost da će biti neizbježno koristiti kameru s digitalnim video izlaznim formatom, poput HDMI, DisplayPort, SDI i sl. kojeg je potrebno pretvoriti u neki od podržanih ulaznih formata računala, najčešće PCIe ili USB, koristeći Capture Card. Kao i svaki elektronički uređaj, CC unosi latenciju te ima ograničenje količine podataka koju može obraditi u jedinici vremena, o čemu treba voditi brigu. Osim te dvije primarne specifikacije, mogućnosti suradnje s DMA sklopom CPU i/ili GPU i eventualna pred obrada signala unutar CC mogu biti od velike pomoći. Na papiru ovo zvuči jednostavno, no pronalazak jasnih, nedvosmislenih podataka pokazao se izazovnim. Neke kartice oglašavaju „low latency“ mogućnosti, no proizvođači ne pružaju nikakve konkretne podatke [36]. Drugi pak navode podršku za GPU SGDMA, no driveri i podrška za novije OS nije jasna [37].

Datapath VisionSC-UHD2 rješava te upitnike, navodeći „single line of capture latency“, rezolucije do 8k H * 8k W, niz formata signala (RGB10, 8-8-8, Y410, 4:2:2, YUY, NV12, YV12...) i neograničen framerate [38]. Navedena je podrška za Windows 10, za kojeg postoje i Media Foundation driveri, ne samo DirectShow, te za Linux, ali nije specificirano za koju verziju. Spominje se i „10-bit processing pipeline“, no nije navedeno što on uključuje. Jedino potencijalno ograničenje je PCIe Gen3 x8 koji pruža 6 GB/s preko oba HDMI sučelja, što znači 3 GB/s individualno. Proizvođač tvrdi da je preko toga moguće izvesti 4K60p na 4:4:4, što je i limit HDMI 2.0 sučelja kojeg koristi, tako da 8K30p na 4:2:0 s njegovih 24.48 Gb/s taman uspijeva proći [39].

Tu je i Magewell Pro Capture HDMI 4K Plus LT koji jasno navodi podršku za nove OS-ove, uključujući Windows 11 (DirectShow API driveri, Media Foundation ne) i Ubuntu 18.04 (V4L2 i ALSA API). Također, detaljnije su opisane mogućnosti dvostrukog 720 Mpix/s 10-bit video cjevovoda. Podržane operacije su obrezivanje (engl. cropping), skaliranje, de-interlacing, promjena H/W omjera signala (engl. aspect ratio), konverzija formata boja, broja sličica u sekundi i čak direktni prikaz videa na zaslonu (engl. On-screen display, OSD). Podržava manje video formata, no veći nedostatak od toga je 4K144p ograničenje, koje uz PCIe Gen2 x4 (2 GB/s) znači da jedna ova kartica primjerice

ne bi zadovoljila 8K zahtjeve [40]. Srećom, sklopovlje i pripadni SDK podržavaju rad s više od jedne kartice pa uz eksterni demuxing videa to se ograničenje može zaobići. Dapače, uz potrošnju od 8W, tri takve CC postižu usporedive rezultate s jednom 22W VisionSC-UHD2, osim po latenciji, koja je u najbržoj konfiguraciji 64 video linije [41]. Proizvođač eksplicitno navodi podršku za AMD DirectGMA i Nvidia GPUDirect te općeniti SG-DMA, pa je teško odrediti koja od dvije kartice ima bolje end-to-end latencije [42].

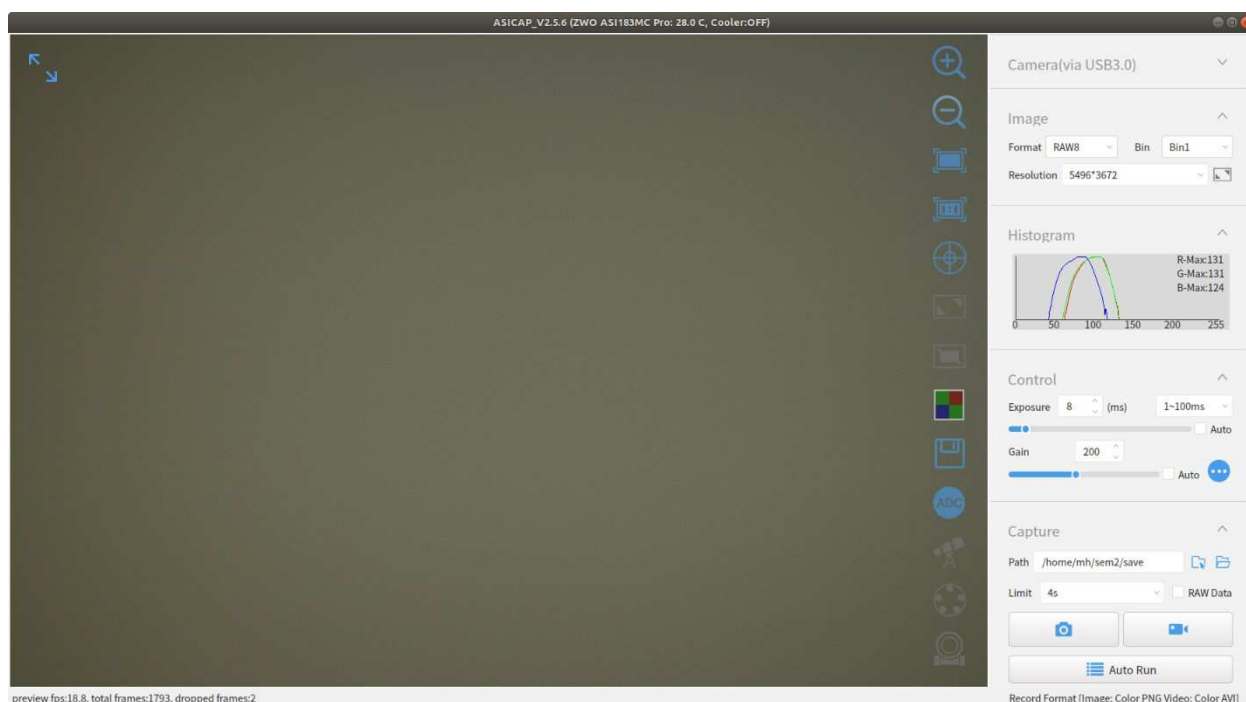
Može se još napomenuti Blackmagicdesign DeckLink 8K Pro (SDI sučelje), Active Silicon FireBird 4xCXP12-3PE8 (CoaXpress sučelje), te AJA Video System, BitFlow, bluefish444, Deltacast, Matrox, Rhode&Schwarz, SiliconSoftware i Yuan kao firme koje proizvode CC s GPUDirect podrškom [41]. Dio njih fokusira se na industrijske i machine learning primjene, stoga su rezolucije u rangu 32k*65k i mogućnosti usporedivog raspona na raspolaganju.

8 Proučavanje konkretne kamere

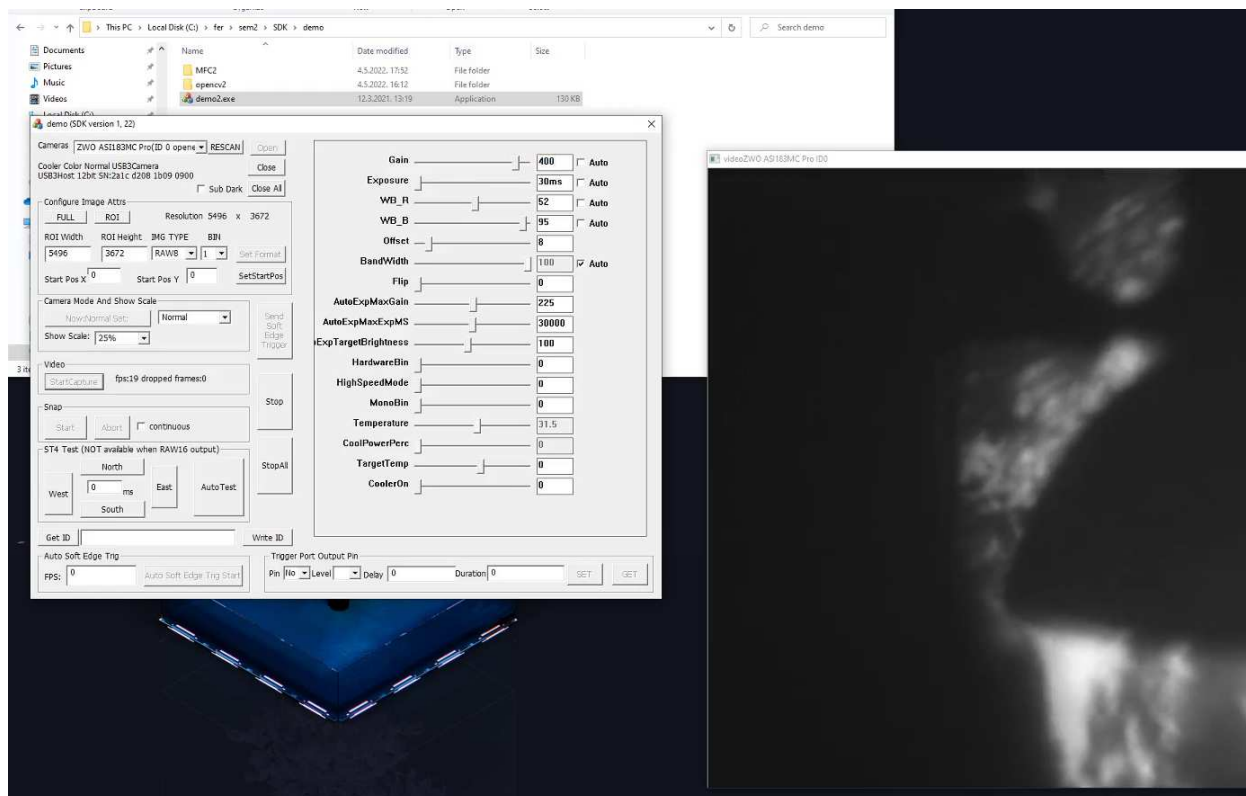
U svrhu boljeg upoznavanja s tematikom te dobivanja okvirnih parametara ulaznog člana izabrana i testirana je konkretna kamera: ASI183MC Pro proizvođača ZWO. Ona spada u eksterno hladjene, odnosno temperaturno stabilizirane, kamere za fotografiranje objekata dubokog neba (engl. deep-sky objekt, DSO) u koje spadaju zvjezdana jata, maglice, galaksije i slično, no zahvaljujući tome što nudi 20.18 MP rezolucije, RAW izlaz te direktno upravljanje i prijenos podataka „u živo“ preko USB 3.0 sučelja pokazala se kao dobar kandidat za naš sustav. Pri punoj rezoluciji od 5496 s 3672 piksela korištenog Sony IMX183CQJ-J senzora te 3 kanala u boji, svaki dubine do 12 bita, kamera je u stanju pružiti 19 sličica u sekundi, odnosno 12.86 Gbps prometa [46]. Još se može napomenuti da kamera ima integriran USB 2.0 kontroler s dva porta čime se povećava fleksibilnost korištenja i spajanja s dodatnom opremom, ugrađeno aktivno hlađenje te mogućnost sklopovskog spajanja piksela (eng. pixel binning) čim se djelomično može rasteretiti CC ili enkoder [45]. Još jedna ključna činjenica je da ZWO besplatno distribuira knjižice, zaglavlja te demonstrativni kod za svoje kamere, uz već gotova Windows, Linux i Mac programska rješenja za upravljanje kamerom i pristup podacima. Ovo znatno olakšava integraciju kamere u naš sustav uz očuvanje oglašanih performansi kamere.

Spomenuti kôd koristi OpenCV. OpenCV (engl. Open Source Computer Vision Library) je popularna biblioteka otvorenog koda koja pruža alate i funkcionalnosti za obradu slika. Olakšava razvoj aplikacija koje se bave računalnim vidom, prepoznavanjem objekata, analizom slika i videa te strojnim učenjem te pruža visoku brzinu izvršavanja i efikasnost koristeći optimizirane algoritme, SIMD instrukcije procesora i paralelizam [57]. OpenCV je prigodan za rad u stvarnom vremenu, i podržava više operacijskih sustav, što kôd čini prenosivim. Ostatak kôda je relativno intuitivan, izuzev kineskih komentara, te osim instalacije i podešavanja putanja za spomenuti OpenCV ne zahtijeva dodatno prilagođavanje Linux okoline. Ukoliko se koristi Windows, potrebno je preuzeti Visual Studio s MSVC, MFC i ATL dodacima za C++ i usmjeriti PATH varijablu okoline prema preuzetim .dll bibliotekama. Bez većih modifikacija moguće je birati između TIFF, PNG i JPG formata ukoliko se pohranjuju individualni okviri ili AVI ako se snima video stoga povezivanje s većinom daljnjih alata za obradu slike ne iziskuje dodatnu obradu. U

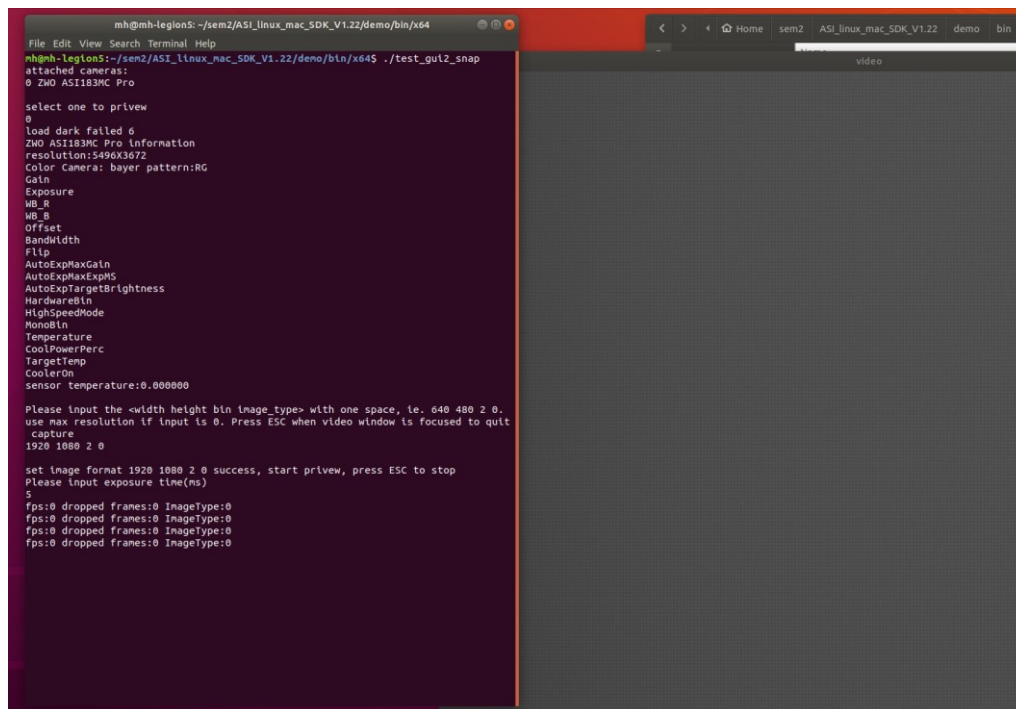
nastavku su priložene tri slike: 6. pokazuje ASISudio alat, 7. grafičko sučelje aplikacije koja demonstrira mogućnosti ASI upravljačke knjižice, dok se u 8. može vidjeti rad programa posvećenog isključivo snimanju.



Slika 6 Linux ASISudio



Slika 7 Windows SDK GUI



Slika 8 Linux video stream skripta

Nakon upoznavanja s kodom, izveden je niz testova s ciljem utvrđivanja okvirnih performansi koje se mogu očekivati uz korištenje opisane kamere i programske potpore. Glavni parametar koji nas interesira je kašnjenje, odnosno latencija cjelokupnog ulaznog člana. Preciznije, zanima nas koliko je vremena potrebno od trenutka kada se okine ekspozicija nove slike (trenutak A), do trenutka kada kamera zaprimi i krene izvršavati taj zahtjev (trenutak B), potom dok se konačna slika ne obradi u kameri (trenutak C) i konačno, dok se ne pročita i dodatno obradi na računalu, kada postaje spremna za pohranu ili prosljeđivanje u naredne članove sustava (trenutak D). Period od A do B manje nam je interesantan jer kamera podržava kontinuirano uzimanje uzoraka (engl. continuous mode), što znači da će se to vrijeme utrošiti samo kod pokretanja sustava, dok će se naknadne slike moći čitati brzinom kojom ih kamera može proizvesti. Kamera, te pripadna upravljačka knjižica, omogućuju uvid u broj slika koje računalu nije stiglo obraditi pa su posljedično izgubljene (engl. dropped frames) te se pokazalo da oba testna računala posjeduju dovoljno računalne moći da do ovoga u praksi nije dolazilo. Odnosno, prijenosno sučelje, radna memorija, procesor te grafička kartica brže su preuzimali, obrađivali i prikazivali slike nego ih je kamera mogla proizvesti. Zbog toga pak možemo zaključiti da je period B-C dulji od perioda C-D, barem na testnom sklopovlju (mašina 1: Ryzen 7 3700X, 16GB DDR4 3200, GeForce GTX 1070 TI 8G, mašina 2: Ryzen 7 4800H, 16GB DDR4 3200, GeForce GTX 1650 4G). Period B-C za N-tu sliku obavlja se paralelno periodu C-D za N-

1. sliku, što znači da je glavni ograničavajući faktor cijelog ulaznog člana period C-D, tj. brzina same kamere.

Preciznije, kašnjenje ulaznog člana obrnuto je proporcionalno broju sličica koje kamera može proizvesti u jedinici vremena te u idealnim uvjetima za maksimalnu rezoluciju iznosi 52.63 ms. To se kašnjenje naravno može smanjiti ukoliko je prihvatljivo smanjiti rezoluciju ili zanemariti boju, ali jednako tako može i rasti ukoliko je potrebno snimati u uvjetima slabog osvjetljenja i popratno produljiti vrijeme ekspozicije koje se linearno pribraja temeljnom kašnjenju. Snimljene brzine za niz konfiguracija kamere, izražene u sličicama u sekundi, priložene su u Tablici 1, dok su zajednički parametri testiranja: HW bin1 (osim ako je drugačije navedeno), Fast Mode i 1ms vrijeme ekspozicije.

Tablica 1 Performanse testirane kamere

Rezolucija	RAW8 (10bit)	RAW16 (12bit)		
	Službeni rezultati		Vlastiti rezultati	
	USB 3.0		USB 2.0	USB 3.0
5496 x 3672	19	19	2	10
3840 x 2160	41.04	36.12	3	23
2560 x 1440	-	-	5	53
1920 x 1080	80.10	70.48	9	72
1920 x 1080 (bin2)	-	-	9	81
1280 x 720	117.30	103.23	-	-
1024 x 768 (bin3)	-	-	23	107
640 x 480	169.92	149.53	-	-
320 x 240	308.17	271.19	-	-

Zbog poteškoća prilagodbe kôda na OpenCV4 biblioteku, testiranje nižih rezolucija nije bilo moguće, no s obzirom na to da želimo ostvariti prijenos slika visoke rezolucije ti su podaci manje bitni. Također, iako je korišteno testno sklopovlje slično po pitanju računalne snage, na slabijoj od dvije mašine zapažen je relativni pad performansi u iznosu od 2,459%. No, navedene rezultate i računala koja stoje iza njih nije uputno uzimati kao referentne, odnosno iz njih donositi općenite zaključke o potrebnim mogućnostima sklopovlja koje će obavljati ulogu enkodera jer je u ovom testu obavljana dosta rudimentarna obradu slike, tj. radilo se o eventualnom skaliranju, rotaciji i sličnim jednostavnim operacijama nad ulaznim podacima. Također, OpenCV se smatra jednim od

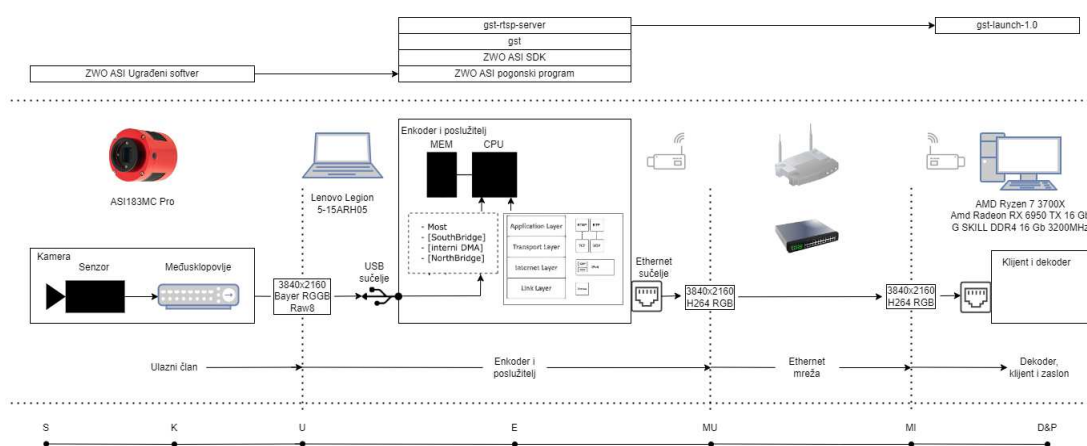
bržih i zrelijih knjižica za grafičku obradu, što ne možemo uzimati zdravo za gotovo, ali se u ovom testu u potpunosti oslanja na CPU za svoje zadatke što, kao što je diskutirano u poglavlju 4., nije optimalno za obradu slike s niskim kašnjenjem.

9 Prvi prototip SPS-a

Do sada smo definirali SPS, sustav čija je svrha mrežni prijenos video signala visoke rezolucije na kratke udaljenosti te sa što kraćim kašnjenjem s kraja na kraj, željene karakteristike istog, od čega se on u teoriji sastoji i kako bismo birali konkretne komponente s kojima bismo ga sastavili. Prva od njih, kamera, je odabrana, nabavljena i testirana, a sada je vrijeme učiti isto s ostatkom sustava. Prvi prioritet ovog prototipa bio je uspjeh izvedbe, a tek onda performanse pa su preferirane gotove biblioteke i rješenja.

9.1 Tijek razvoja

Kako bi se lakše objasnio tijek rada, na slici X je prikazana ažurirana blok shema SPS-a. Od vrha prema dnu: prikazan je pojednostavljeni programski stog, navedena imena korištenih komponenti, pa formati i sučelja u lancu te konačno označene ključne točke koje svaka slika mora proći kako kasnije bilo lakše opisati kašnjenja pojedinih komponenti. Slika u većem formatu priložena je uz rad.



Slika 9 Dijagram prvog prototipa SPS

9.2 Veza dva računala

Prijenosno računalo u ulozi enkodera te stolno računalo u ulozi dekodera povezani su u LAN mrežu preko preklopnika. Definirana su i potrebna pravila pristupa na prijenosnom računalu na kojem je Ubuntu 22.04 i podešen vatrozid (engl. firewall) stolnog računala s

Windows 10 Pro operacijskim sustavom. Ispravnost konfiguracije testirana je 'netcat' alatom. Sljedeći je korak bio kodirati i poslati video s jednog računala na drugo. U tu svrhu korišteni su OBS Studio, LIVE555 Media Server te VLC Media Player i pomoću njih formiran inicijalni prototip SPS-a. O korištenim protokolima i obradi videa bit će rečeno kasnije, a ovdje bih samo istaknuo dva glavna:

- Real Time Streaming Protocol (RTSP) zadužen za multipleksiranje, paketizaciju i uspostavljanje sjednice za dijeljenje multimedijskog sadržaja, u ovom slučaju videa. Temelji se na TCP-u i omogućava upravljanje tokom s klijentske strane preko naredbi poput „play“ i „pause“
- Real-time Transport Protocol (RTP) koji u stvari prenosi podatke dijeljenog sadržaja i temelji se na UDP-u.

LIVE555 Media Server je aplikacija bazirana na C biblioteci istih autora i njena je zadaća prijenos medijskih datoteka u ulozi RTSP poslužitelja. Izgrađena inačica za Windows podržava niz kontejnera te audio i video formata, od kojih je u moje svrhe poslužio MPEG Transport Stream kontejner (.ts sufix) u kojeg je multipleksiran samo jedan H.264 video tok. Posao stvaranja istog odradio je OBS Studio snimajući zaslon računala. Razmjena podataka odrađena je preko datoteke jer je to predviđeni princip rada unaprijed izgrađenih inačica oba programa. Na klijentskom računalu je zadatak čitanja RTSP/RTP toka i njegovog prikaza na zaslonu obavio VLC Media Player. Ova probna inačica sustava nudila je poprilično loše performanse, s preko jedne sekunde kašnjenja i s rezolucijama manjim od 4K, ali je potvrdila izvedivost sustava.

9.2.1 Odabir formata i protokola za prijenos

Kako bismo mogli odabrati koji protokol će se koristiti za prijenos video signala, prvo što treba odlučiti je hoće li se sadržaj iz kamere slati kao niz slika, kakav se i čita, ili će se iz njega formirati neki oblik video toka.

Slanje niza slika ima nekoliko prednosti, od kojih je najbitnija smanjenje kašnjenja u određenim uvjetima. Kada se formira video, primjerice po MPEG-4 standardu, potrebno je akumulirati nekoliko slika i na pošiljatelju i na primatelju, što očito unosi kašnjenje koje nije prisutno ako se pojedine slike šalju čim su spremne. No, točno kašnjenje i duljina obrade nisu fiksno određeni. Primjerice, spomenuti MPEG-4 standard, točnije njegova inačica 10 poznatija kao H264 omogućuje podešavanje intenziteta procesa kodiranja

uključujući i uklanjanje tzv. P i B okvira koji zahtijevaju akumuliranje. Konkretnije, ako je to prihvatljiv kompromis, uz korištenje samo jedne predviđene B slike uz svaku „punu“ I sliku, potrebna brzina prijenosa uz istu kvalitetu prikaza gotovo se prepolovljuje. Također, ako je bitna mogućnost oporavka od greške, cijela se slika po potrebi može ponovno poslati, što nije tako lako kada govorimo o videu.

Uzimajući u obzir da prednosti slanja niza slika postaju manje s porastom broja sličica u sekundi te činjenicu da su pokoji izgubljeni okvir i smanjena vizualna kvaliteta prihvatljivi, prednosti slanja niza slika u ovom slučaju nisu presudne. S druge strane, algoritmi za komprimiranje videa mogu postići veće stupnjeve kompresije uklanjajući vremensku redundanciju, te, zahvaljujući svojoj podesivosti, olakšavaju prilagođavanje toka prema trenutnim uvjetima u komunikacijskom kanalu [47]. Zbog toga je u konačnici odlučeno je koristiti video tok.

Nakon što je utvrđen zahtjev za slanje video toka preko LAN mreže, potrebno je odabrati tehnologiju, standard, odnosno protokol koji to omogućava. Pronađeno je njih mnogo, no svaki je zamišljen za neku konkretnu uporabu pa je proučavanje i izbor nešto jednostavniji. Konkretnije, ovo su neki od protokola za dijeljenje videa grupirani po njihovoj namjeni:

- Komunikacija u stvarnom vremenu: RTSP/RTP, WebRTC, WebSockets, QUIC (Quick UDP Internet Connections)
- Pouzdana komunikacija preko javnog interneta: SRT, HLS, MPEG-DASH, Zixi, V-Nova PERSEUS
- Profesionalna i studijska okruženja: SDI, NDI, AMWA NMOS

Prva grupa fokusira se na nisko kašnjenje, druga na rad preko šire Internet infrastrukture, što obuhvaća gledanje videa u preglednicima preko HTTP-a, podršku za segmentaciju, prijenos u živo i na zahtjev itd. Treća pak grupa uzima u obzir sinkronizaciju, centralizirano upravljanje sustavom, prijenos nekomprimiranog sadržaja i slične faktore važne za profesionalna TV i film okruženja. U kontekstu SPS-a evidentno je najinteresantnija prva grupa, čiji su protokoli detaljnije opisani u nastavku.

WebSockets omogućava obostranu komunikaciju što nama nije potrebno, dok je QUIC transportni, a ne aplikacijski protokol. Odabirom jednog od njih nužno bi porasla složenost SPS-a, pa se fokusiramo na RTSP/RTP te WebRTC. Između ta dva, WebRTC je bliži karakteristikama i funkcionalnostima koje SPS zahtjeva, primarno jer obećava manje kašnjenje naspram RTP-a, ali ima i nekoliko mana: dolazi s ugrađenim mehanizmima

kompenzacije gubitaka paketa i varijacije kašnjenja za koje nije sigurno mogu li se onemogućiti. Dalje, predviđen je za rad u web preglednicima koji su nešto sporiji od namjenskih programa poput gst-launch-1.0, ffmpeg, VLC i sličnih. I konačno, WebRTC je relativno nova i slabije zastupljena i dokumentirana tehnologija što otežava razvoj i znači da postoji veći rizik da SPS ne bude realiziran. S druge strane, RTSP/RTP, usprkos nešto većoj složenosti, podržan je u većem broju uređaja, biblioteka i programa, zna raditi s mnogobrojnim kodecima i formatima, nudi i mogućnost dinamičkog podešavanja brzine prijenosa (engl. Adaptive bitrate streaming), već spomenuto upravljanje reprodukcije s klijenta pa je u konačnici odabran za ostvarenje SPS-a.

9.3 Snimanje i prijenos pravog videa

9.3.1 Spajanje prave video kamere na računalo

Sljedeći je korak bio povezati pravu kameru na sustav, odnosno prenositi aktivnu snimku. Za to je pak potrebno pristupiti kameri te preuzeti i modificirati izlazni videosignal u format prigodan za ostatak sustava. U tu svrhu mogao sam koristiti pripremljeni program proizvođača kamere pod nazivom ASISudio, no tu postoje dvije nepovoljne činjenice: složena programska potpora unosi dodatno i neželjeno kašnjenje što nije opravdano dodatnim mogućnostima koje nudi. I drugo, razmjena podataka između te aplikacije i ostatka sustava morala bi se odraditi preko datoteka što također donosi nepredvidivo i osjetno kašnjenje [48]. Zbog toga je u konačnici odabrana alternativna opcija: razvoj vlastite aplikacije temeljene na pogonskim programu kamere te kompletu za razvoj softvera (engl. SDK) dostupnom na stranici proizvođača. Većina ovog posla obavljena je u sklopu 8. poglavlja, stoga je taj kôd samo trebalo proširiti ili povezati s onim za kodiranje te RTSP/RTP prijenos.

9.3.2 Formiranje toka iz pravog videa. Odabir arhitekture, biblioteka i alata.

Rezultantni kôd ovog koraka prošao je niz iteracija zahvaljujući povećem broju permutacija dostupnih komunikacijskih protokola, biblioteka i aplikacija koje ih implementiraju te internoj arhitekturi drugog člana na slici X. Prva dilema nastala je pri odabiru aplikacijske arhitekture. Po završetku prototipa SPS-a u 9.2 prva pomisao bila je samo izmijeniti početni član lanca, odnosno, koristeći ASISudio, pohranjivati izlaz

kamere kao niz bitmap slika. Potom bi se taj niz kodirao u video tok i dalje na jednak način slao preko RTSP/RTP. No, kao što je rečeno, dodatni troškovi sada dvostruke pohrane i čitanja datoteka ne podudaraju se sa zahtjevom niske latencije stoga su analizirane druge opcije.

Naime, kako bi se omogućilo korištenje danas dostupnih bežičnih tehnologija za prijenos toka, ako ne odmah onda u nekoj budućoj inačici, potrebno je taj tok komprimirati. Primjerice, koristeći nominalnu brzinu najmodernijeg WiFi standarda WiFi 6 koja iznosi 9.6 Gbit/s i izmjerenu brzinu generiranja podataka korištene kameru od 8.8 Gbit/s pri maksimalnoj rezoluciji naizgled i nije potrebna kompresija. No, u taj račun nisu uključena zaglavlja, kodiranje i slični fiksni troškovi WiFi standarda, odnosno to nije njegova korisna, neto brzina koja iznosi nešto skromnijih 1.2 Gbit/s i zahtjeva kompresiju izvornog toka faktorom 7,33. Koristimo li pak dostupniji WiFi 5 (802.11ac) i njegovu prosječnu neto brzinu od 0.8 Gbit/s, originalni tok trebali bi komprimirati faktorom 11, a koristeći nešto stariji WiFi 4 (802.11n) prosječne neto brzine od 0.1 Gbit/s faktorom 88 [52][53]. Korištenjem žične veze, točnije 10 GigE ili bržeg standarda, koji pruža brzinu prijenosa 10 Gbit/s, mogli bismo slati video signal bez kompresije, no ništa od autoru dostupne opreme ne podržava taj standard. Lokalna LAN mreža je unaprijeđena s 100 Mbit/s standarda na 1 Gbit/s i na njoj su izvedeni svi testovi.

S obzirom na to da je kompresija neizbježna, istraženi su alati koji ju mogu pružiti i to po mogućnosti u kontekstu prijenosa uživo. Neiscrpna lista pronađenih uključuje: FFmpeg, Gstreamer, DirectShow, Media Foundation, MJPEG Tools i druge. Želio bih naglasiti da su ovo holistički alati, odnosno programski okviri koji obuhvaćaju niz biblioteka za razne oblike video obrade, ne samo kodiranje, iza unificiranog sučelja. Cilj ovog projekta mogao bi se postići i korištenjem pojedinih većih biblioteka poput OpenCV, VideoCodec C++, oneVPL++, libavcodec, libavformat, ali i specijaliziranih: OpenH264, libx264, libx265, libvpx i sl.

I sada, uzimajući u obzir već diskutirano izbjegavanje rada s datotekama korištenjem radnog spremnika u kontekstu vlastite aplikacije izgrađene povrh SDK za rad s kamerom, uspostavljanje lokalnog RTSP/RTP poslužitelja te formiranje i kodiranje video toka, ponovno su sagledani istraženi postojeći alati i biblioteke tu su identificirane dvije opcije:

- korištenje diskretnih alata za svaki od ta tri zadatka gdje bi razmjena podataka bila obavljena koristeći neki od mehanizama dostupnih u operacijskom sustavu. Ovdje

se primarno misli na redove poruka, cjevovode i dijeljenu memoriju, gdje su po svojim karakteristikama druga dva bolji odabir, uz preporuku korištenja neke vrste duplog ili kružnog spremnika.

- objedinjenje svih funkcionalnosti u jednu aplikaciju gdje bi razmjena podataka mogla biti jednostavnije izvedena.

Prva opcija omogućuje veću fleksibilnost, odnosno takva bi izvedba olakšala testiranje većeg broja tehnologija i implementacija jer pojedine komponente nisu čvrsto vezane. Tako bi se primjerice korištena kamera mogla lako zamijeniti s drugim izvorom slike. Ili bi se pak dobiveni video tok lakše mogao prenositi alternativnim aplikacijama i protokolima ili drugim implementacijama istog. Gledajući samostalne implementacije RTSP poslužitelja pronađeno je njih mnogo: FFmpeg, Darwin Streaming Server, Mischa Spiegelmock's RTSP server, gst-rtsp-server, Red5 Pro, Live555, VLC, Adobe Flash Media Server, Wowza te mnogi drugi bazirani na njima, poput rtsp-simple-server i revmischa rtsp-server.

No, s druge strane, veći broj diskretnih komponente diže složenost sustava te zahtjeva dobro poznavanje navedenih mehanizama komunikacije među procesima ako se oni žele optimalno iskoristiti. Zbog toga je odlučeno koristiti drugi pristup i pronađena su dva alata koja to omogućuju: FFmpeg i Gstreamer. Odabir jednog pratio je slične argumente kao i odabir RTSP/RTP nad WebRTC te je odabran Gstreamer, preciznije biblioteke gst i gst-rtsp-server.

9.3.3 Implementacija

Razvoj kôda bio je inkrementalan i iterativan. Izrađeno je 8 inačica aplikacije, svaka s 3 do 8 inkremenata, gdje su testirani razni pristupi generiranja, učitavanja, pohrane, formatiranja, kodiranja te slanja podataka. Testirano je građenje cjevovoda za obradu videa ručnim instanciranjem, konfiguriranjem i povezivanjem komponenti te stvaranjem istog koristeći alate za parsiranje argumenata zadanih u tekstualnom formatu. Varirane su i uloge korištenih biblioteka i komponenata. Naime, SDK pogonskog programa koristi OpenCV biblioteku za internu obradu slike, a iste alate nudi i Gstreamer pa su izrađene verzije aplikacije koje se više oslanjaju na jednu, odnosno drugu. Uz čitanje pravih slika s kamere moguće je i generirati nasumične slike unutar aplikacije, koristiti postojeće elemente Gstreamer biblioteke (videotestsrc) te čitati datoteke s diska.

Nakon zasebnog uhadavanja kôda za generiranje, kodiranje i posluživanje slike, sve tri komponente integrirane su zajedno s pogonskim programom i na kraju čine jednu C++ aplikaciju. S obzirom na to da je izbor konačnih alata i biblioteka dotjerivan tijekom cijele implementacije, komponente za kodiranje i posluživanje postoje i u inačicama gdje se koristi FFmpeg C API za kodiranje, odnosno Live555 biblioteke za posluživanje, no one nisu integrirane s konačnom aplikacijom.

9.3.4 Dekodiranje i prikaz

Za dekodiranje i prikaz toka na klijentskom računalu testirana su tri alata: gst-launc-1.0, VLC Media Player te ffmpeg. U konačnici je prevladao prvi zbog lakog konfiguriranja, minimalnog kašnjenja i dostupnosti na oba korištena operacijska sustava.

9.4 Tehničke značajke

U ovom poglavlju kratko će biti prokomentiran kôd konačne aplikacije te iznesena i interpretiranja kašnjenja u sustavu u ovisnosti o parametrima snimanja i kodiranja.

9.4.1 Aplikacija

Među definicijama u aplikaciji bitna za istaknuti je sljedeća:

```
#define STREAM_DONTDRAW 1
```

Pomoću nje odabire se hoće li program iscrtavati dobiveni signal iz kamere na zaslonu ili će ga proslijediti prema RTSP poslužitelju. Razlika tih dvaju vremena predstavlja kašnjenje koje unosi put po mreži, odnosno period između točaka MU i MI na slici 9.

Isječak 1 predstavlja funkciju zaslužnu za punjenje slike u Gstreamer cjevovod gdje se ona onda obrađuje i kasnije šalje prema poslužitelju

```

1.    // Push IplImage* data stored in global GAsyncQueue *queue
    to GstElement *appsrc;
2.    static gboolean timeout (MyContext * ctx)
3.    {
4.        ...
5.        // fetch data
6.        IplImage* iplImg = (IplImage*)g_async_queue_pop(queue);
7.        cv::Mat img = cv::cvarrToMat(iplImg);
8.        guint8* data = img.data;
9.
10.       /* create a new buffer */
11.       buffer = gst_buffer_new_memdup(data, SIZE);
12.
13.       /* increment the timestamp */
14.       GST_BUFFER_PTS (buffer) = ctx->timestamp;
15.       GST_BUFFER_DURATION (buffer) =
gst_util_uint64_scale_int (1, GST_SECOND, FPS);
16.       ctx->timestamp += GST_BUFFER_DURATION (buffer);
17.
18.       /* push the buffer into the pipeline */
19.       ret = gst_app_src_push_buffer (GST_APP_SRC (appsrc),
buffer);
20.       ...
21.       return TRUE;
22.    }

```

Kôd 1 Punjenje Gstreamer cjevovoda slikama

Aplikacija u sebi sadrži dvije dretve: prva je zaslužna za čitanje podataka iz kamere te učitavanje istih u globalnu GAsyncQueue strukturu koja ima ugrađenu kontrolu pristupa te blokira dretve ukoliko pokušaju čitati ako je prazna ili pisati ako je puna. Pročitana slika se tada omata u cv::Mat klasu s kojom je lakše baratati te se iz nje kreira GstBuffer međuspremnik koji se šalje u cjevovod. Slikama se dodaje i vremenska oznaka kako bi se mogle ispravno kodirati i poslati. Treba napomenuti da se ova funkcija poziva kada je spremna nova slika, u tzv. „push“ režimu rada, no moguće je podesiti i da radi u tzv. „pull“ režimu gdje se poziva kada klijent zatraži novu sliku.

Za ispravno kodiranje i prikazivanje signala potrebno je navesti njegov format što radi sljedeći isječak:

```

gst_object_set (G_OBJECT (appsrc), "caps",
gst_caps_new_simple ("video/x-raw",
"format", G_TYPE_STRING, "RGB",

```

```

"width", G_TYPE_INT, WIDTH,
"height", G_TYPE_INT, HEIGHT,
"framerate", GST_TYPE_FRACTION, FPS, 1, NULL), NULL);

```

Parametar format automatski se podešava prema odabranim postavkama kamere, što znači da će za Raw8, Raw16 i RGB24 biti postavljen kao GRAY8, RGB, odnosno GRAY16_LE.

Pri pokretanju poslužitelja stvara se i spomenuti cjevovod za obradu i to prema proslijeđenom tekstualnom opisu. Kako bi se ubrzao razvoj, odnosno izbjeglo ponovno upisivanje parametara pri svakom pokretanju, oni su u trenutnoj inačici fiksno postavljeni u kôdu i izgledaju ovako:

```

1.    gst_rtsp_media_factory_set_launch (factory,
2.        "( appsrc name=mysrc ! videoconvert ! video/x-
raw,format=(string)I420 ! "
3.        "timeoverlay valignment=4 halignment=1 ! "
4.        "x264enc speed-preset=ultrafast tune=zerolatency
bframes=0 key-int-max=1 ! "
5.        "rtph264pay config-interval=1 name=pay0 pt=96
        )" );

```

Druga linija definira trenutnu aplikaciju kao izvor podataka te identifikator `mysrc` preko kojeg se taj ulazni element dohvaća. Nakon toga se u cjevovod smješta element koji izvodi poduzorkovanje RGB video signala u 4:2:0 signal. Treća linija dodaje štopericu na originalni signal kako bi se olakšalo mjerenje kašnjenja, no više o tome u sljedećem poglavlju. Četvrta linija definira koder i njegove postavke. U ovom primjeru koristi se H264 koder iz x264 biblioteke, podešen da prioritizira brzinu na uštrb kvalitete, odnosno efikasnosti. Parametar `tune=zerolatency` gasi B okvire te povećava broj I okvira naspram P okvira. Parametri `bframes` i `key-int-max` prikazani su samo kao primjer, a pomoću njih može se ručno podesiti isto ponašanje kojim upravlja i `tune` parametar. Konačno, peta linija dodaje i konfigurira RTSP „utovarivač“ (engl. `payload`), odnosno element čiji je zadatak pretvoriti video tok u niz RTP paketa. Podešen je tako da pakira H264 video tok te istu informaciju uključuje u RTP pakete postavljajući `pt` postavku na 96 što označuje H264 video. `config-interval=1` parametar traži da se SPS i PPS NAL polja prenose uz svaki okvir, što ubrzava vrijeme potrebno dekomeru da reagira na

promjene u formatu toka, primjerice rezolucije, te da lakše barata s promjenjivim vremenskim oznakama.

I konačno ovdje je kôd kojim se pokreće RTSP poslužitelj:

```
1.    gst_init (&argc, &argv);
2.
3.    // init thread safe message queue
4.    queue = g_async_queue_new();
5.
6.    // start server
7.    GThread* server_thread = g_thread_new("server",
gststreamer_server, NULL);
```

Preko parametara proslijeđenih programu inicijalizira se gst biblioteka. Potom se inicijalizira i spomenuti globalni red za razmjenu slika između dretvi te pokreće do sada pokazani Gstreamer kôd u novoj dretvi.

U ostatku kôda od korisnika se traže postavke kamere prema kojima se ona onda i konfigurira te se čitaju i prosljeđuju slike iz kamere koristeći spomenutu `GAsyncQueue` strukturu.

9.4.2 Kašnjenje pojedinih komponenti te s kraja na kraj

Jedna od ključnih motivacija ovog projekta bila je utvrditi izvedivost SPS-a te uspostaviti referentnu točku za performanse istog koristeći komercijalno dostupan hardver i softver. U ovom poglavlju bit će prikazana razna kašnjenja konačnog prototipa, no prvo par riječi o tome što se točno i na koji način mjeri.

Gledajući dužinu na dnu slike 1 vidimo točke koje redom predstavljaju: Scenu, Kameru, Usb izlaz kamere, Enkoder, Mrežni ulaz, Mrežni izlaz te Dekoder i Prikaz. Te su oznake definirane kako bi se lakše označila razna interesantna kašnjenja u sustavu.

Tsp - Kašnjenje s kraja na kraj

Ukupno vrijeme potrebno da se promjena u sceni propagira kroz cijeli SPS i prikaže na zaslonu. Isto će biti izmjereno snimanjem cijelog SPS-a eksternom kamerom visoke brzine, nakon čega će biti utvrđen redni broj sličice na kojoj se dogodila promjena u S te

broj sličice u kojoj se ta promjena vidi na D. Razlika tih brojeva podijeljena s brojem sličica u sekundi eksterne kamere dat će traženi Tsp.

Tku - Latencija kamere

Duljina obrade je faktor izvan naše kontrole, odnosno fiksna je i ovisi o korištenoj kameri. Kao takvog korisno ga je znati prije izvođenja optimizacija dijelova sustava jer se primjerice može pokazati da on čini značajan dio ukupnog kašnjenja, ili je sam po sebi dulji od proizvoljno zadanog maksimalnog Tsp. U tom slučaju ćemo pa znamo da moramo nabaviti bržu kameru. Ovo vrijeme najbolje možemo aproksimirati kao inverznu vrijednost maksimalnog broja sličica u sekundi koje kamera može isporučiti jer znamo da kamera nema interni cjevovod, odnosno međuspremnik za obradu slika. Primjerice, ako kamera šalje 100 sličica u sekundi i pod pretpostavkom da se niti jedna ne izgubi, što znamo jer pogonski program ima detekciju izgubljenih sličica, onda je latencija kamere jednaka $1/100 = 10 \text{ ms}$.

Tue - kašnjenje USB prijenosa

Detalji kašnjenja raznih danas korištenih računalnih sučelja, uključujući ovdje korištenog USB3.0, izneseni su u prijašnjim poglavljima te su identificirana alternativna sučelja koja obećavaju bolje performanse. Tue će se izmjeriti korištenjem unaprijed generiranih slika kao ulaz u video cjevovod čime će se utvrditi Tep pa Tue možemo dobiti kao Tsp-(Tku+Tep), ako će preciznost korištenih alata to dozvoliti.

Tm - trajanje mrežnog prijenosa

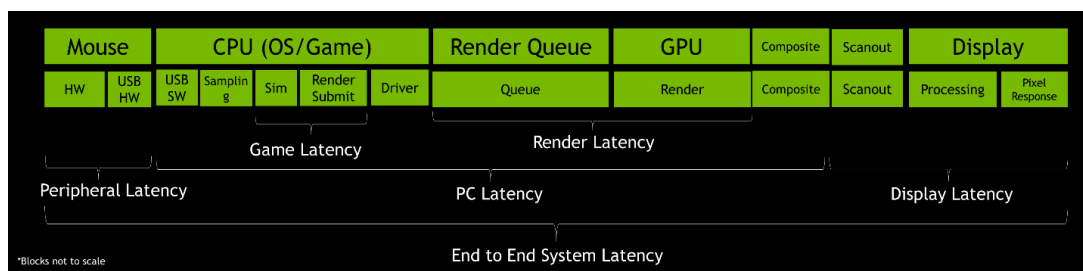
Ovo bi vrijeme bilo korisnije da su računala povezana bežičnom vezom, ali poznavanjem njega moći ćemo odrediti duljinu same obrade, odnosno Tem. Mjerenje možemo izvesti korištenjem ping naredbe s prilagođenim paketima koji nalikuju na one koje šalje naša aplikacija

Tem - trajanje kodiranja

Ovom ćemo mjerom najbolje moći utvrditi utjecaj raznih postavki cjevovoda, odnosno utvrditi cijenu raznih koda, njihovih opcija, konfiguracija GOP-ova i sl. Utvrđivanje Tem nije trivijalno jer iako vrijeme kada slika postane dostupna u memoriji možemo odrediti ispisom trenutka u kojem je završila funkcija za dohvat, vrijeme kada nezavisna Gstreamer dretva pošalje tu istu sliku u mrežu nije tako jednostavno.

- Primjer eksternog pristupa mjerenju: na monitoru prikazujemo sat visoke rezolucije te ispišemo indeks pročitane slike i taj isti indeks upišemo u samu sliku. Kada se slika s praćenim indeksom iscrta na ekranu usporedimo trenutak ispisa i trenutak iscertavanja. Dobivenoj razlici treba dodati kašnjenje ispisa jer toliko kasni ispis indeksa od trenutka kada je slika dostupna. Potrebno je kompenzirati i latenciju monitora te kašnjenje iscertavanja. Konačni rezultat odgovara Tem.
- Interni pristup mjerenju: konfiguriramo Gstreamer da obrađuje i šalje okvire što brže može čime dobivamo da je izlazna frekvencija okvira obrnuto proporcionalna Tem. Potom u međuspremnik za slike učitamo niz slika jedne boje, pa N nasumično generiranih slika i konačno niz slika druge boje. Tada možemo izmjeriti period trajanja prikazivanja N slika i tu vrijednost podijelimo s N kako bismo dobili Tem.

Složenost utvrđivanja Tem utemeljena je kompleksnosti modernih operacijskih sustava i sklopovske arhitekture što je ilustrirano na slici 10.



Slika 10 Iscertavanje na zaslon kod modernih računala [54]

Izvorno, ova slika opisuje latenciju računala u kontekstu videoigara, odnosno koliko je vremena potrebno da se korisnički unos iscrta na zaslonu. No, isti principi vrijede i za SPS, samo što u našem slučaju „igru“ predstavlja naša aplikacija. Vidimo da, čak i ako bi se naša aplikacija izvodila kao jedna dretva čije trajanje možemo odrediti uspoređujući vrijeme početka i završetka, slika koju pokušavamo iscertati mora proći niz programskih sučelja, dodatnu obradu i sinkronizirati se s periodom iscertavanja na zaslon. Treba napomenuti da OpenCV u korištenoj konfiguraciji ne koristi grafičku karticu za iscertavanje, no neki od Gstreamer elemenata mogu izvoditi kodiranje na njoj. Primjerice, umjesto x264enc koda biblioteke x264 možemo koristiti nvh264enc koder koji također može provoditi H.264 kodiranje, ali koristi NVENC API tvrtke NVIDIA koji iskorištava resurse grafičke kartice [51]. Trenutno, osim H.264, NVENC API nudi i H.265 te AV1 kodere i H.264, H.265, MJPEG, MPEG-1/2/4, VP8/VP9, VC-1 te AV1 dekodere.

9.5 Rezultati prvog prototipa SPS

U nastavku nalaze se tablice s rezultatima mjerenja. Na početku su ispitane razne konfiguracije kamere uz fiksne postavke aplikacije. Potom su fiksirane postavke kamere, a variran Gstreamer cjevovod.

Vrijednosti Tsp uz variranje rezolucije kamere i maksimalne frekvencije uzorkovanja Gstreamer dretve prikazane su u tablici 2.

- postavke kamere: vrijeme ekspozicije: 1ms, bin: 1, format: RGB24
- postavke cjevovoda: „x264enc speed-preset=ultrafast tune=zerolatency ! rtph264pay config-interval=1 pt=96"

Tablica 2 Kašnjenje s kraja na kraj u ovisnosti o rezoluciji snimanja i frekvenciji Gst dretve

Rezolucija [pikseli]	Ograničena frekvencija uzorkovanja Gstreamer dretve [Hz]	Tsp [ms]
1024 x 720	-	112,5
2560 x 1440	-	3537,5
3840 x 2160	-	8350
3840 x 2160	10	166,66
3840 x 2160	20	170,83
3840 x 2160	30	1891,66

Vrijednosti Tsm uz variranje maksimalne frekvencije uzorkovanja Gstreamer dretve prikazane su u tablici 3.

- postavke kamere: vrijeme ekspozicije: 1ms, bin: 1, format: RGB24
- postavke cjevovoda: „x264enc speed-preset=ultrafast tune=zerolatency ! rtph264pay config-interval=1 pt=96"

Tablica 3 Kašnjenje od scene do mreže u ovisnosti o frekvenciji Gst dretve

Rezolucija [pikseli]	Ograničena frekvencija uzorkovanja Gstreamer dretve [Hz]	Tsm [ms]
3840 x 2160	10	125
3840 x 2160	20	212,5

Vrijednosti Tsm uz variranje postavki kodera prikazane su u tablici 4.

- postavke kamere: vrijeme ekspozicije: 1ms, bin: 1, format: RGB24, rezolucija: 3840 x 2160
- ograničena frekvencija uzorkovanja Gstreamer dretve [Hz]: 20
- postavke cjevovoda: „x264enc tune=zerolatency ! rtph264pay config-interval=1 pt=96 ”

Tablica 4 Kašnjenje od scene do mreže u ovisnosti o postavkama kodera

speed-preset	tune	Tsm [ms]
ultrafast	zerolatency	125
fast	zerolatency	729,16
normal	zerolatency	854,16
ultrafast	-	1354,16
fast	-	2837,5
normal	-	3358,33

Vrijednosti Tsm uz variranje korištenog kodera i pripadnog utovarivača prikazane su u tablici 5.

- postavke kamere: vrijeme ekspozicije: 1ms, bin: 1, format: RGB24, rezolucija: 3840 x 2160
- ograničena frekvencija uzorkovanja Gstreamer dretve [Hz]: 20

Tablica 5 Kašnjenje od scene do mreže u ovisnosti o korištenom utovarivaču

Koder	Utovarivač	Ograničena frekvencija uzorkovanja dretve [Hz]	Tsm [ms]
x264enc	rtph264pay	20	1354,16
x265enc	rtph265pay	20	4741,66
vp9enc	rtpvp9pay	10	-
vp8enc	rtpvp8pay	10	2150

av1enc	rtpayload	10	-
--------	-----------	----	---

Vrijednosti T_{em} uz variranje rezolucije prikazane su u tablici 6.

- postavke kamere: vrijeme ekspozicije: 1ms, bin: 1, format: RGB24
- postavke cjevovoda: „x264enc speed-preset=ultrafast tune=zerolatency ! rtpH264pay config-interval=1 pt=96"

Tablica 6 Kašnjenje od scene do mreže u ovisnosti o rezoluciji snimanja

Rezolucija [piksli]	Ograničena frekvencija uzorkovanja Gstreamer dretve [Hz]	T_{em} [ms]
1024 x 720	1000	3,83
1920 x 1080	1000	13,04
3840 x 2160	1000	-

Kašnjenja čija vrijednost ne ovisi o gore mijenjanim parametrima:

- Mrežno kašnjenje, dobiveno kao razlika T_{sp} i T'_{sp} , gdje T'_{sp} predstavlja kašnjenje izmjereno kada se slika iscrivavana odmah nakon kodiranja, je $T_m = 67,5$ ms
- Mrežna latencija između dvaju računala izmjerena je pomoću alata hrPING koji podržava rezoluciju mjerenja od 1us te je kroz 10000 mjerenja utvrđen prosječan RT od 0.434us (stddev=0.158us).
- Latencija Gstreamer cjevovoda formiranog prema „x264enc speed-preset=ultrafast tune=zerolatency ! rtpH264pay config-interval=1 pt=96" argumentima, a dobivena mjerenjem vremena potrebnog da se iscrivaju unaprijed generirane slike, iznosi $T_{gs} = 3,9$ ms

9.6 Komentari

Imajući na umu da je proizvod ovog projekta prvi prototip složenog sustava te da u konačnici nisu izvedene ekstenzivne optimizacije, ukupno kašnjenje s kraja na kraj od 166,66 ms za prostornu rezoluciju od 4K pri 10 slićica u sekundi sluti na dobro. Također, široka razdioba dobivenih vrijednosti T_{sp} i T_{sm} pokazuje koliki utjecaj ima konfiguracija

cjevovoda za obradu videa, posebice odabir koda i njegovih postavki. Imajući na umu i različite arhitekture SPS-a diskutirane u 4. poglavlju te mogućnost korištenja koprocesora, snažnijeg mrežnog sklopovlja i sl., ostvarenje inicijalnih zahtjeva u budućnosti čini se izgledno.

Vrijednosti kašnjenja za VP9 i AV1 kodere nisu izmjerena. AV1 koder uklonjen je iz osnovne Gstreamer biblioteke te je procijenjeno da je vrijeme potrebno da se pronađe i uhoda alternativa bolje uloženo u postojeći prototip. S druge strane, VP9 koder je isproban, no nije bilo moguće dobiti suvisle vrijednosti kašnjenja za korištene parametre. Proces kodiranja trajao je pre dugo te bi došlo do zagušenja sustava unutar prve sekunde odašiljanja. Sličan je simptom uočen je i kod ostalih koda kada je brzina toka narasla na određenu razinu, primjerice kod pokušaja mjerenja Te pri 4K rezoluciji. No, razlozi iza ovoga trenutno nisu poznati te je potrebno dodatno istraživanje prije nego se može reći nešto više.

Valja i komentirati veliku razliku između performansi x264 te x265 koda. H.265, poznat i kao HEVC, novija je i efikasnija norma od H.264 te je i predviđen za visoko-rezolucijske standarde poput UHD TV pa bi se moglo očekivati da će davati bolje performanse u kontekstu SPS-a. No, treba imati na umu da efikasnost H.265 proizlazi iz složenije obrade, koja je i više nego red veličine dugotrajnije obrade u usporedbi s H.264, stoga se u sustavima koji zahtijevaju nisku latenciju, što će reći i brzu obradu, i dalje zna koristiti H.264 (Minopoulos, 2020). Sličnom zaključku vode i gore izneseni podaci.

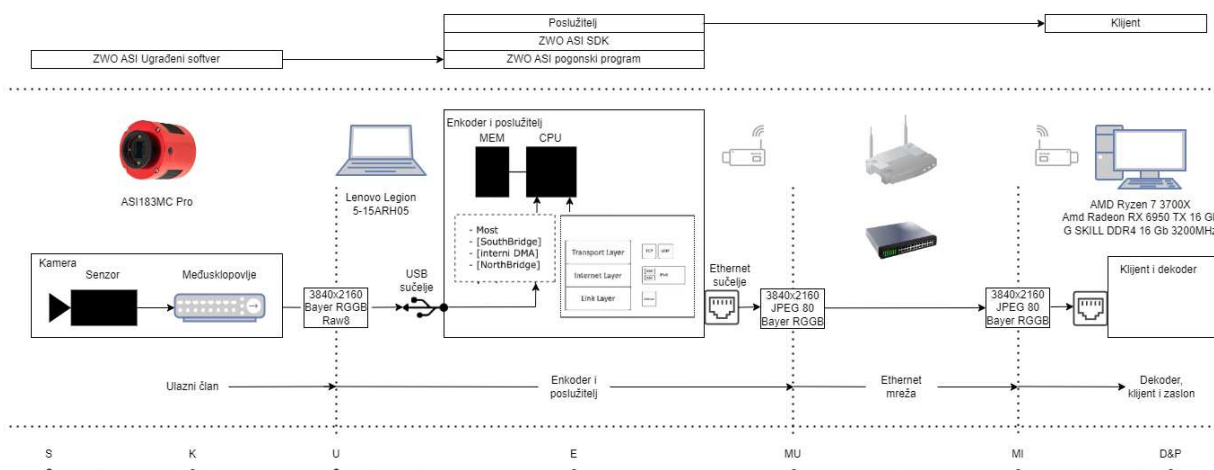
Napor koji je bio potreban kako bi se identificirali i istražili dostupni alati potrebi za realizaciju ovog projekta nadmašio je inicijalna očekivanja. Iako razvijeni prototip postiže cilj realizacije cijelog SPS-a, postoji volja i potreba za dodatnim testiranjem već korištenih, ali i novih koda te njihovih parametara, razvijanje naprednijih metoda mjerenja kašnjenja, bolje iskorištavanje trenutnog sklopovlja itd. U tu svrhu bit će potrebno intimnije poznavanje primijenjenih alata, mrežnih tehnologija te rada s video tokovima općenito, a potencijalno i proučavanje, izrada te razvoj heterogenog sustava koji bi bio prilagođeniji zadatku kojim se ovaj sustav bavi.

10 Drugi prototip SPS-a

Završetkom prvog prototipa utvrđena je praktična izvedivost SPS-a i dobivene su inicijalne vrijednosti kašnjenja pojedinih komponenata sustava te ukupno kašnjenje s kraja na kraj. Kao konačni korak ovog rada preostaje iskoristiti ta saznanja kako bi se sustav dalje optimizirao, odnosno dobivena kašnjenja minimizirala i to je svrha drugog prototipa. U tu svrhu bit će napravljen korak nazad, ponovno sagledane dostupni protokoli, algoritmi i rješenja te implementirana nova programska podrška.

10.1 Novi pristup

U prijašnjem poglavlju rečeno je da su format slanih podataka i arhitektura aplikacije odabrani na način da konačno rješenje nudi veću fleksibilnost, a time i da ostvarenje željene funkcionalnosti bude jednostavnije. Odabran je video tok umjesto niza sličica te su korištene složene aplikacije kako bi se izradile pojedine komponente sustava. Na taj je način također izbjegnuta potreba za izradom klijentske aplikacije, jer je u tu svrhu moguće koristiti postojeće programe za reprodukciju videa preko mreže. Taj se pristup pokazao uspješnim jer je bilo moguće brzo uspostaviti funkcionalan sustav i modificirati njegovo ponašanje, ali ima i par mana, a glavna među njima je kompleksnost konačne aplikacije. Složenost konfiguracije cjevovoda za obradu videa i razumijevanje njegovog rada predstavljali su veliki prepreku razvoju i optimizaciji sustavu, a nedostatak kontrole nad pojedinim elementima cjevovoda pokazao je da će za postizanje boljih performansi biti potrebno ažurirati neke od inicijalnih pretpostavki. Konkretno, odlučeno je napustiti formiranje video toka i pokušati prenijeti individualne slike što je brže moguće, a to najefektivnije postiže uklanjanjem postojećeg aplikacijskog protokola i prijelazom na direktno slanje datagrama preko UDP priključnice. Blok dijagram s istim sklopovskim komponentama, no ažuriranim formatima i aplikacijskim stogom je na slici 11.



Slika 11 Dijagram drugog prototipa SPS

10.2 Poslužitelj i klijent

Cjelovit izvorni kôd poslužitelja i klijenta može se pronaći u sljedećem [Git repozitoriju](#), a ovdje će biti opisani dijelovi bitni za performanse i rezultate.

Kôd poslužitelja podijeljen je u tri dretve: glavna, dretva poslužitelja i crtaća dretva. Glavna dretva se prva pokreće, podešava kameru, potom pokreće druge dvije i čita slike iz kamere. Dretva poslužitelja uspostavlja UDP vezu s klijentom, uzima pročitane slike iz reda i redom ih šalje klijentu. Crtaća dretva pomaže glavnoj dretvi u pretovaru slika iz inicijalnog međuspremnik u red te iscrtava sliku na zaslonu poslužitelja, ako se tako odabere. Ovakva organizacija osigurava da čitanje i slanje slika ne ometaju jedno drugo, odnosno prijenos i mrežni uvjeti neće uzrokovati gubitak slika u kameri i obratno.

Slično tome, kôd klijenta podijeljen je u tri dretve: glavna dretva koja uspostavlja UDP vezu i pokreće druge dvije, dretva koja prima pakete i sastavlja slike od njih te dretva za obradu i prikaz slika. I u klijentu je bilo bitno izvesti paralelizaciju jer je tok paketa gotovo konstantan pa ako bi čitanje sljedeće slike izvodili nakon procesiranja i prikaza, značajan broj paketa bi bio izgubljen.

Nakon pokretanja oba programa, prvi korak je UDP „rukovanje“ kojim poslužitelj sazna adresu klijenta i tijekom kojeg se razmjene lokalna vremena oba uređaja. Naime, u ključnim trenucima oba programa ispisuju trenutna vremena i to je jedan od načina kako je mjereno kašnjenje, pa je bitno ustanoviti razliku njihovih satova. Kako bi se ona minimizirala, prije pokretanja ažurirani su sistemski satovi preko Windows Time service na Windows klijentu i `timedatectl` na Ubuntu poslužitelju, a razlika satova u rasponu

mikrosekundi kompenzira se utvrđivanjem mrežnog kašnjenja između računala i korištenjem Cristianovog algoritma. Uz prosječnu izmjerenu devijaciju kašnjenja od 180us, zaključeno je da je ova metoda dovoljna precizna, a konačna dobivena kašnjenja su potvrđena i drugim metodama mjerenja, stoga nema sumnje u njihovu točnost.

Osim biblioteke i zaglavlja koja služe za upravljanje kamerom, u aplikaciji su korišteni već spomenuti OpenCV, ovdje korišten za obradu i rukovanje slikama, te Boost. Boost je biblioteka koja pruža dodatne funkcionalnosti i poboljšanja standardne C++ biblioteke, poput rada s nizovima, datotekama, komunikaciju putem mreže i slično. Njena glavna svrha u ovom programu je podrška za mrežni rad, a odabrana je prije drugih biblioteka zbog svojih performansi, zrelosti i podrške za više platformi, ključno Windows i Linux, što je uvelike olakšalo razvoj.

10.2.1 Konfiguracija rada

Ponašanje poslužitelja i klijenta da se znatno modificirati i to je izvedeno preko niza definicija kojima se dalje, pomoću pred-procesorskih naredbi, uključuje i isključuje prevođenje dijelova kôda. Ovaj pristup kôd čini potencijalno nešto manje razumljivim, ali pomaže pri optimizaciji. Primjer bitnijih postavki na poslužitelju je odabir hoće li se slike komprimirati ili slati kao niz vrijednosti piksela, koristimo li slike iz kamere, nasumično generirane slike ili unaprijed pripremljene slike na disku, koja je brzina slanja i sl. Na klijentu je moguće konfigurirati hoće li se provoditi interpretacija Bayer formata (engl. debayering), koja metoda paralelizacije se koristi za obradu slike, izvode li se korekcije boje itd.

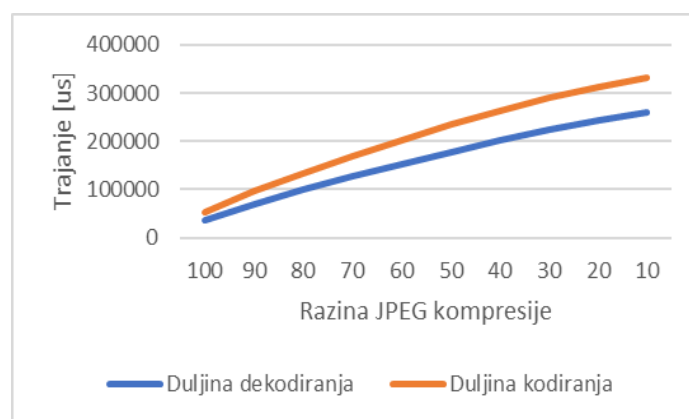
Variranjem ovih postavki moguće je podesiti broj slika koje poslužitelj šalje u sekundi, opterećenje mreže, broj slika koje klijent može procesirati te kašnjenje, pa će u poglavlju s rezultatima biti navedene korištene vrijednosti ovih postavki.

10.2.2 Čitanje i kompresija slika

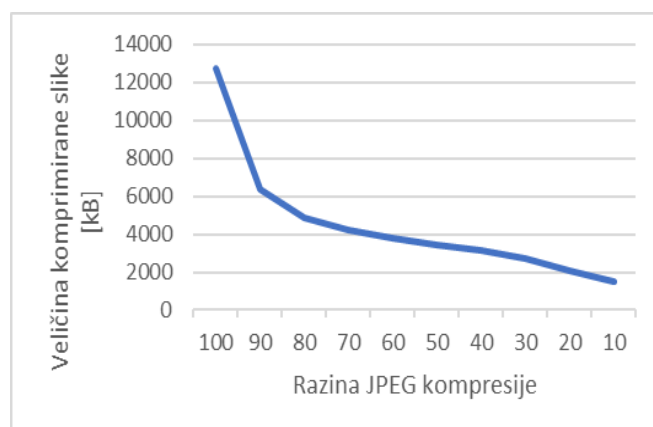
Čitanje slika izvodi glavna dretva poslužitelja preko ASIGetVideoData metode, kada je izvor kamera, i taj zadatak zahtjeva 9.5% korištenih CPU resursa. Ovo nije blokirajući poziv, odnosno ne izvodi radno čekanje, ali je i dalje postavljen u zasebnu dretvu kako ga ne bi ometali drugi zadaci. Kad se provodi mjerenje kašnjenja želi se osigurati ponovljivi

uvjeti pa se tada koriste unaprijed pripremljene slike koje su po pokretanja programa učitaju u memoriju. Ovisno o odabranim postavkama, slike se potom komprimiraju.

Kao što je spomenuto u poglavlju 9.3.2, brzina podataka našeg video toka nadilazi mogućnosti konkretno korištene mrežne infrastrukture pa je i u drugom prototipu korištena kompresija. Proučeni su generički algoritmi kompresije, te algoritmi s i bez gubitaka predviđeni za slike i međusobno uspoređeni po pitanju računalne kompleksnosti, odnosno brzine te faktora kompresije. JPEG norma pokazala kao najbolji odabir, ne samo radi relativno male kompleksnosti u usporedbi s drugim normama već i zbog svoje podesivosti, što olakšava maksimizaciju iskorištenja lokalne mreže [56]. U idealnom slučaju, želimo utrošiti minimalno moguće vremena i resursa na komprimiranje, održati određenu razinu kvalitete slike, ali i dalje komprimirati u toj mjeri da ne zagušimo mrežu. Kako bi se odredilo koji stupanj kompresije donosi najveću uštedu s najmanjim utjecajem na izgled, testirani su stupnjevi kompresije, od 100 do 10, u inkrementima po 10 na bijelom šumu 4K rezolucije, a rezultati su prikazani na slikama 12 i 13.

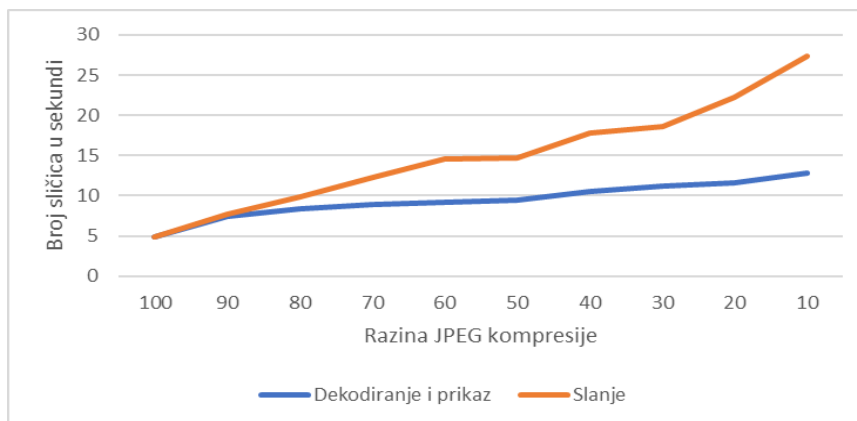


Slika 12 Duljina kodiranja i dekodiranja u ovisnosti u stupnju JPEG kompresije



Slika 13 Veličina komprimirane slike a u ovisnosti u stupnju JPEG kompresije

Prema očekivanju, veličina slika pada s porastom stupnja kompresije, ali uz to raste i trajanje kodiranja i dekodiranja. Zbog toga je bilo potrebno utvrditi do koje mjere se isplati kodirati, a to je odrađeno mjerenjem brzine cijelog sustava u ovisnosti o razini kompresije. Rezultati su prikazani na slici 14.



Slika 14 Brzina sustava u ovisnosti o razini JPEG kompresije

Iz perspektive brzine sustava, osjetljivo je veća korist većeg faktora kompresije, stoga jedini ograničavajući faktor na iznos kompresije predstavlja kvaliteta slike i tu je subjektivno odlučeno da će se u testovima koristiti razina kompresije 80, no ta se vrijednost lako prilagodi mogućnostima korištene mrežne infrastrukture.

Testirani su i drugi parametri kodiranja. Prvi je progresivno kodiranje, koje sliku kodira u više prolaza što poboljšava krajnji izgled, ali zahtjeva više vremena te može povećati veličinu datoteke. Drugi je optimizacija Huffmanovih kodnih tablica što smanjuje veličinu datoteke, ali naravno iziskuje vrijeme. Točni iznosi su izmjereni i prikazani u tablici 7.

Tablica 7 Utjecaj dodatnih parametara kompresije na veličinu slike i performanse sustava

	Uštedeni prostor [%]	Redukcijski faktori	
		Brzina kodiranja	Brzina dekodiranja
Progresivno kodiranje	6,1717	2,7244	3,1458
Optimizacija Huff. tablica	4,2644	1,1223	1,3074
Oboje	6,1717	2,8556	3,2357

Navedene uštede veličine nisu se pokazale vrijedne faktora redukcije brzine pa progresivno kodiranje i optimizacija Huffmanovih tablica nisu korišteni.

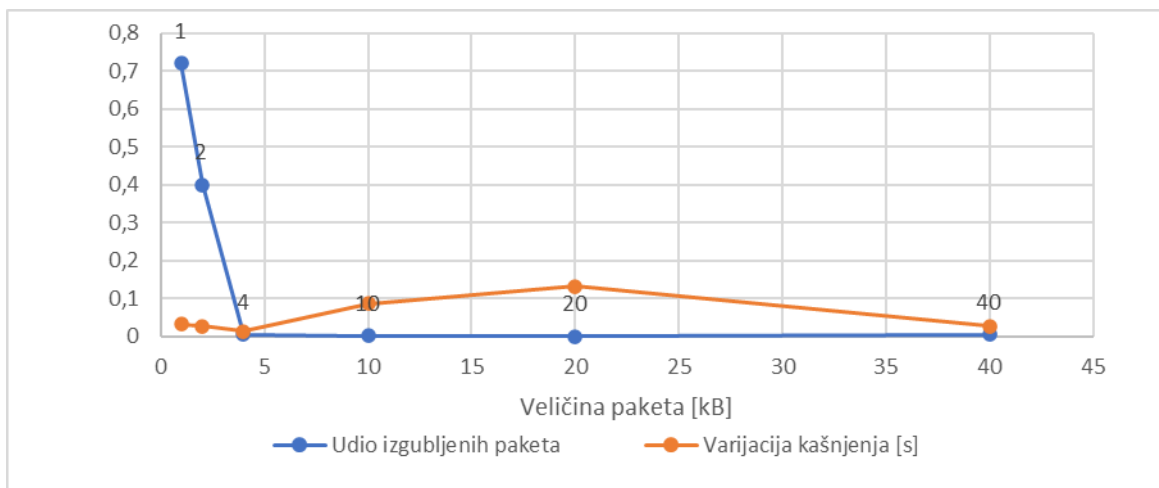
Treći i zadnji parametar kodiranja je period resetiranja kodiranja. On uzrokuje da se u sliku periodički umetnu markeri pomoću kojih se dio slike može oporaviti tijekom dekodiranja ako je došlo do greške prilikom prijenosa. U našim testovima, dodavanje 8 markera po paketu povećalo je prosječnu veličinu slike za 564 B, što je zanemariv iznos kada govorimo o slikama okvirne veličine 1 MB, ali je zato kvaliteta dekodirane slike za primjer bijelog šuma znatno narasla. Kao i za stupanj kompresije, ovaj je parametar najbolje prilagoditi konkretnim uvjetima mreže kako bi se dosege željena kvaliteta dekodirane slike.

Radi konzistentnosti, svi navedeni parametri fiksirani su, odnosno nisu mijenjani kroz daljnje testove.

10.2.3 Pakiranje slika

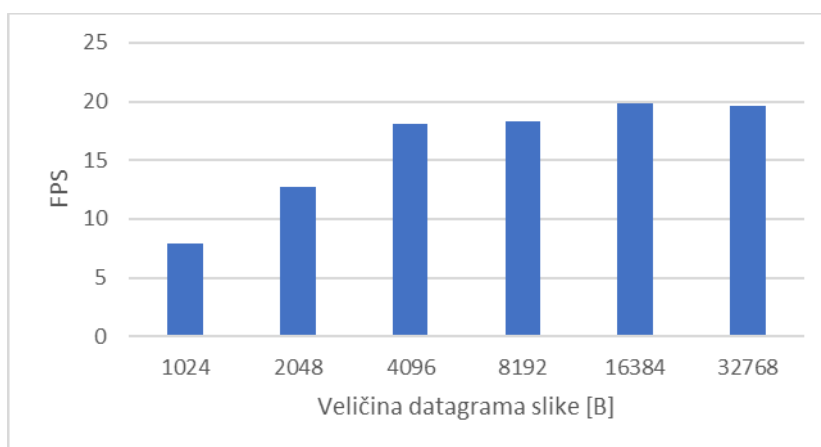
Sadržaj slika pakira se u datagrame podesive veličine, a uz sadržaj šalje se i malo zaglavlje koje sadrži podatke o rednom broju konkretnog paketa u nizu, njegovoj veličini i par osnovnih informacija o cijeloj slici. Te informacije koristi klijent tijekom sastavljanja slike i eventualne nadopune podataka iz paketa koji nisu stigli na odredište. Na kraju slike dodaje se podesiv broj tzv. FIN paketa koji označavaju kraj slike. To se inicijalno čini redundantno jer klijent zna ukupno veličinu slike i u teoriji, pomoću te informacije i brojanja sakupljenih paketa, može zaključiti kada je slanje slike završilo, no ako ispadne i jedan paket klijent će čekati u nedogled. Drugi potencijalna metoda detekcije bila bi da klijent pretpostavi kraj slanja slike N kada primi prvi paket N+1. slike, no to podrazumijeva da N+1 slika uopće dolazi i, bitnije, znači da početak obrade slike N bespotrebno čeka nedefinirano udaljen dolazak sljedeće slike. U konačnici se rješenjem s FIN paketima čini kao najbolja opcija. Tehnički je dosta poslati jedan FIN paket, no kako bismo izbjegli vjerojatnost da upravo on ispadne šalje se njih 3, ali i taj se broj lako može podesiti u ovisnosti o uvjetima mreže.

Optimalna veličina i broj paketa određeni su eksperimentalno. U tu svrhu korišten je 'iperf' alat te su varirani parametri paketa koji se šalju, a ukupna brzina prijenosa pokušala održati na 900 Mbit/s jer stabilnost brzo pada na većim brzinama. Izmjereni rezultati prikazani su na slici 15.

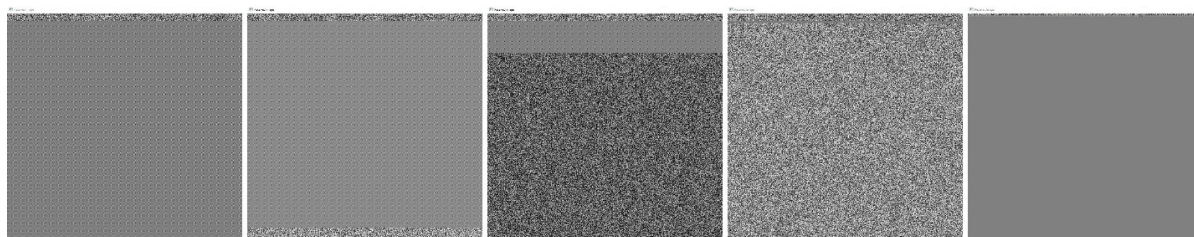


Slika 15 Udio izgubljenih paketa i varijacija kašnjenja u ovisnosti u veličini paketa

Iz njega vidimo da, pri odabranoj brzini, broj izgubljenih paketa naglo raste ako veličina paketa padne ispod 4 kB, odnosno broj paketa naraste iznad 250000 u sekundi. S druge strane, što su paketi veći to više vremena treba da se oni napune, više podataka gubimo ako jedan od njih ne stigne i veći je rizik od gubitka paketa zbog vanjske fragmentacije. Nadalje, tijekom testiranja primijećeno je da variranje veličine paketa utječe i na performanse sustava te kvalitetu slike, o čemu svjedoče slika 16 i 17.



Slika 16 Performanse sustava u ovisnosti o veličini paketa



Slika 17 Utjecaj veličine paketa na vizualnu kvalitetu slike

Kao što je i pretpostavljeno, performanse sustava osjetno pate kada su paketi manji od 4 KB, dok se najbolji rezultati i vizualna kvaliteta postiže uz veličinu paketa od 16 KB. Treba napomenuti da su na slici Y isječki dekodiranih slika veličine 1000 x 1000 piksela te kao takvi nisu nužno reprezentativni cjelokupne vizualne kvalitete, nego priloženi samo za ilustrativne svrhe. Zaključno, odabrana je optimalna veličina od 4 KB. S njom ne premašujemo ograničenje broja paketa u jedinici vremena, niti trpimo znatan pad brzine i kvalitete pri gubitku paketa.

10.2.4 Prihvaćanje paketa

Prihvaćanje paketa se uz pomoć paralelizma i programske arhitekture ne krši s obradom slika, te koristi par prečica s ciljem maksimizacije količine primljenih podataka, uz minimalno dodatno kašnjenje.

Po dolasku paketa prvo se pročita njegovo zaglavlje i donese se odluka je li paket ispravan, što će reći očekivane veličine te pripada li trenutnoj slici, a potom ako se radi o podatkovnom paketu njegov sadržaj spremamo, a ako se radi o FIN paketu pokreće se sastavljanje slike. Ako je došao FIN paket, a nemamo još sve podatke, podaci koji fale pune se zamjenskim vrijednostima i slika se tek onda sastavlja. Ovo se pokazalo nužnim jer u suprotnom dekodiranje slike može baciti grešku i sustav učiniti nestabilnim.

Slika se sastavlja i pohranjuje u globalnu strukturu koja je izvedena kao trostruki međuspremnik: kako nove slike dolaze, naizmjenično se pohranjuju u spremnik A i B, a kada dretva za obradu i prikaz zatraži sljedeću sliku, međuspremnik A ili B, u kojem god se nalazi zadnja potpuna slika, postaje međuspremnik za obradu C, a prijašnji C spremnik zamjenjuje A ili B. Na taj način dretva primanje paketa može neometano i stabilno pohranjivati slike, a dretva za obradu može u bilo kojem trenutku dobiti kompletnu sliku spremnu za obradu i ta slika neće biti starija od dva perioda slanja. Klasa koja se brine o međuspremnicima, BufferManager, također osigurava da međuspremnik u obradi ostane zaključan sve dok dretva obrade ne obavi svoj posao te će blokirati dretvu obrade ako je ona brža od dretve koja sprema slike. U konačnoj verziji programa, primanje paketa zaslužno je za samo 3% od ukupne potrošnje CPU resursa i nikada nije predstavljalo usko grlo u radu.

10.2.5 Obrada slika

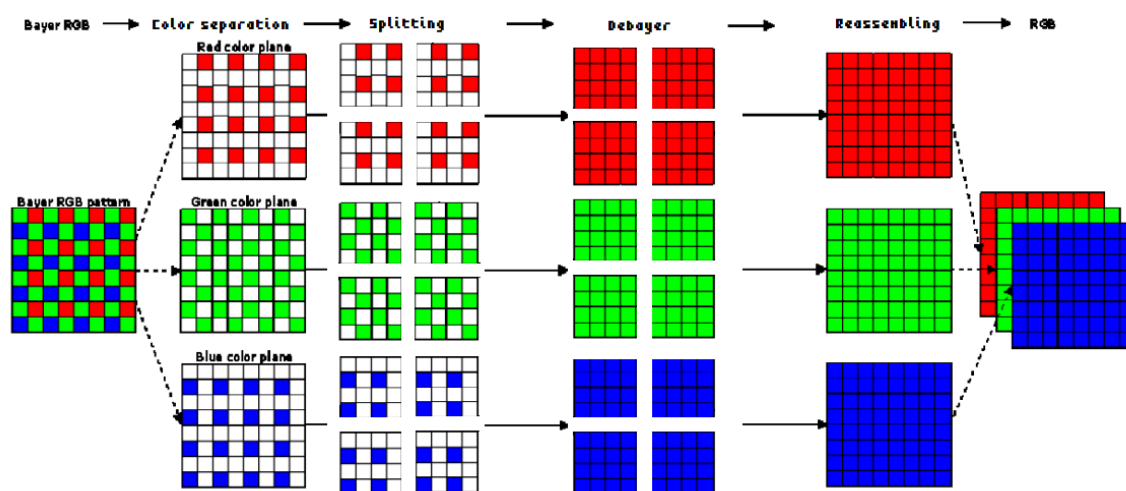
Zadnji korak u radu SPS-a, delegiran u zasebnu dretvu, je obrada i prikaz slike. Redom, tu spada dekodiranje JPEG kompresije, debayering, korekcija boje i svjetline, učitavanje slike u red za prikaz i sam prikaz, a u nastavku ću biti rečeno nešto više o optimizacijama koje su obavljene u svrhu smanjenja kašnjenja.

10.2.5.1 Dekodiranje

Operacije dekodiranja JPEG slike nazad u polje vrijednosti piksela u prosjeku uzima 4,1% CPU resursa i izvodi se pomoću `cv::imdecode()` funkcije. Na rad iste nije moguće utjecati unutar programa, odnosno njena brzina ovisi o internoj implementaciji algoritma za dekodiranje te o svojstvima slike, no OpenCV je izgrađen specifično za sklopovlje na kojem radi i zbog toga uspješno iskorištava svih 16 dostupnih logičkih procesora te njihove SIMD ekstenzije, što maksimizira brzinu dekodiranja. Proces dekodiranja bilo bi moguće ubrzati paralelnom obradom više slika, no to pak zahtjeva uvođenje cjevovoda za obradu ili akumuliranje slika, a oboje unose neželjeno dodatno kašnjenje pa nije iskorišteno.

10.2.5.2 Debayering

Interpretacija Bayer formata, poznatije kao engl. debayering ili engl. demosaicing, obavlja `cv::cvtColor` metoda i ona je naj zahtjevnija operaciju koju klijent izvodi, s prosječnih opterećenjem od 69,66% CPU resursa i zbog toga je najviše vremena uloženo na optimizacije baš nje. OpenCV u pozadini koristi algoritam bilinearne interpolacije, varijabilnog broja gradijenata, obrade uz svjesnost rubova ili obrada s alfa kanalom, ovisno o odabranim parametrima, a za naše je svrhe odabrana bilinearna interpolacija jer u prosjeku naj brža [55]. Proces debayeringa se najlakše opisuje grafički, i možemo ga vidjeti na slici 18.



Slika 18 Proces interpretacije Bayer RGGB formata (engl. debayering) [58]

Ukratko, ulazno polje skalarnih vrijednosti segmentira se prema fiksnom 2x2 uzorku, gdje pikseli od gornjeg lijevog do donjeg desnog redom predstavljaju R, G, G i B kanal. Nedostajuće vrijednosti između njih se interpoliraju i tri novo dobivena polja nazad se spajaju u sada tro-kanalnu, odnosno RGB sliku. Iz perspektive optimizacije, olakotna okolnost je u tome što se sama operacija da lako paralelizirati jer se svodi na nezavisan niz aritmetičkih operacija. To omogućava da se izvorna slika podijeli na dijelove i svaki od njih obradi zasebno ili da se za cijela operacija obavi na grafičkom koprocesoru (GPU).

Podjela na više dijelova unosi vizualne artefakte na njihovim granicama i zahtjeva dodatne kućanske poslova za rastavljanje i ponovno sastavljanje slike, dok obrada u GPU unosi kašnjenje jer se slika mora kopirati iz radne memorije u međuspremnik na GPU i obratno pa su izvedeni testovi kako bi se utvrdio najbolji put dalje. Upravljanje, što uključuje raspoređivanje zadataka po dretvama, sinkronizaciju pristupa dijeljenim resursima i upravljanje životnim ciklusom dretvi obavljeno je na tri načina, i to koristeći:

- Threading Building Blocks (TBB) je biblioteku otvorenog koda, točnije njenu TBB Arena komponentu. Arena je višedretvena struktura podataka koja omogućuje izvršavanje zadataka, tj. jedinica izvršavanja, na više dretvi. TBB pruža automatsko upravljanje dretvama te ju sam OpenCV koristi ako je izgrađen s potrebnim postavkama.
- Boost Thread Pool koji dolazi u Boost biblioteci i olakšava ponovno korištenje dretvi iz slike u sliku, čime se smanjuje količina kućanskih poslova. Kao i TBB,

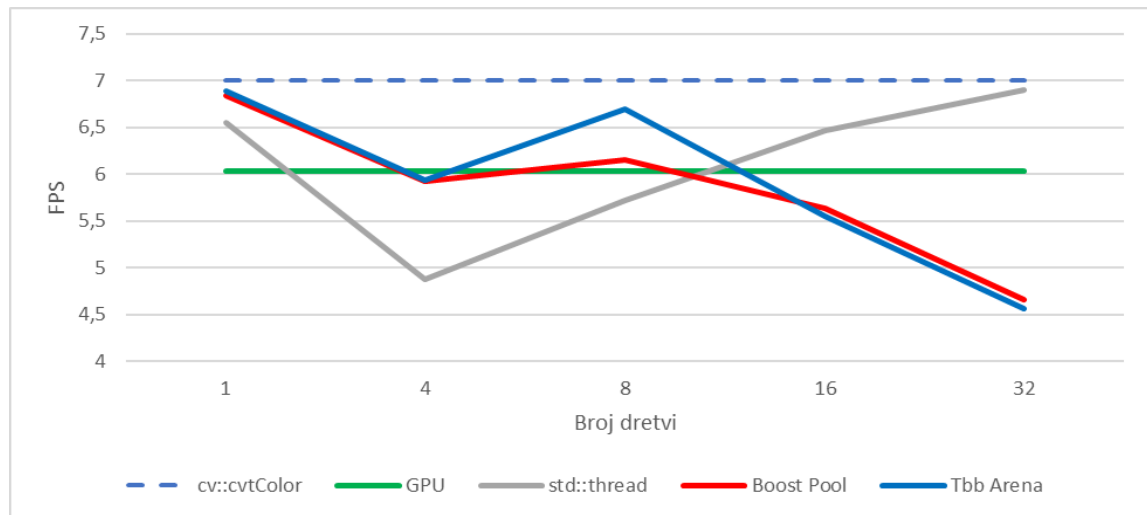
omogućuje automatsko stvaranje grupe dretvi i raspoređivanje zadataka na te dretve.

- Standardnu C++ biblioteku i mogućnosti unutar `std::thread` modula. Ovo je iziskivalo ručno stvaranje i upravljanje dretvama, ali s druge strane, to pak omogućuje potencijalno najefikasniju implementaciju jer imamo potpunu kontrolu nad cijelom operacijom.

Kako bi se minimizirala čekanja na dijeljene resurse i troškovi prebacivanja konteksta dretvi, odlučeno je povezati broj segmenata na koje se slika dijeli te broj dretvi koji na njima radi. Drugim riječima, umjesto da se više dretvi prepire oko istih segmenata, sliku ćemo podijeliti na onoliko dijelova koliko koristimo dretvi, što znači da rad jedne dretve ne utječe na drugu i praktički nemamo dijeljene resurse jer svaka dretva pristupa drugom dijelu slike.

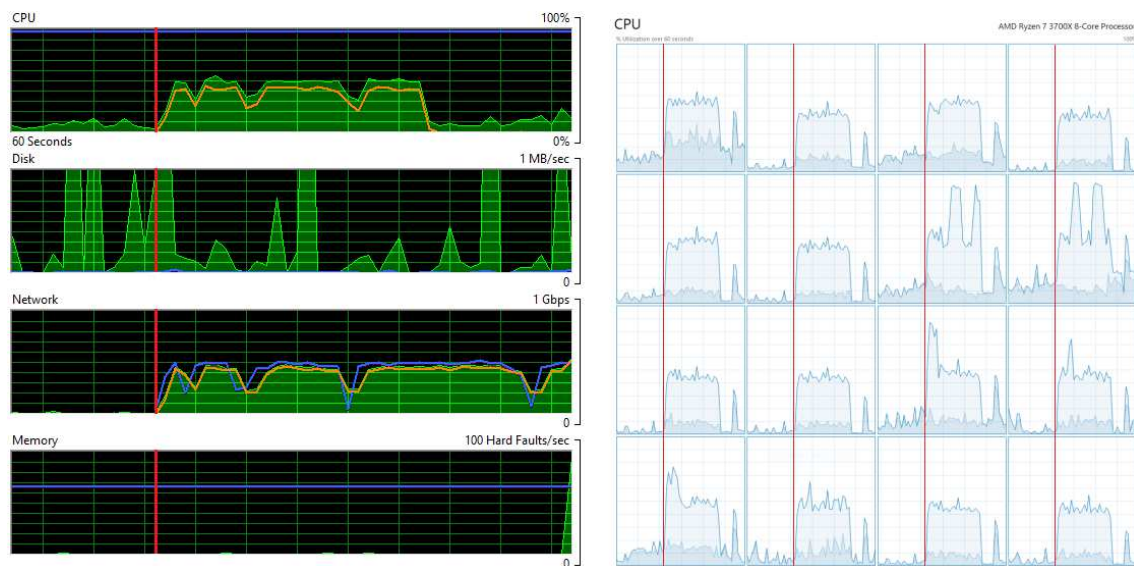
Konačno, za obradu slika pokušao se iskoristiti i GPU. Slutnja je da bi se ovaj pristup mogao isplatiti jer je debayer operacija upravo tip zadatka za kojeg je GPU dizajniran: velik broj jednostavnih, bezuvjetnih, nezavisnih, aritmetičkih operacija nad dugim, neprekinutim nizom podataka. Suprotno inicijalnim slutnjama, ovo je implementacijski bilo lakše za izvesti nego druga tri pristupa jer su u OpenCV-u 3.0 dodane nove „transparentne sklopovske optimizacije“, implementirane u OpenCL-u. Kako bi ih se omogućilo, i dalje se pozivaju iste funkcije iz `cv::` imenskog prostora, a jedina razlika je što se za pohranu slika treba koristiti `cv::UMat` struktura podataka umjesto starije `cv::Mat` strukture. Biblioteka će automatski koristiti GPU optimizacije ako su dostupne.

Sva četiri pristupa uspoređena su s običnim pozivom `cv::cvtColor` nad cijelom slikom i rezultati su na slici 19.



Slika 19 Usporedba performansi raznih pristupa debayeringu

Zanimljivo je primijetiti da su svi pokušaji ubrzavanja debayering procesa paralelizacijom i korištenjem GPU sporiji od onoga što običan poziv metode postiže. Kako bi se odgonetnulo zašto, izmjerena je i količina resursa koju aplikacija koristi uz običan poziv cv::cvtColor metode, i rezultati su prikazani na slici 20.



Slika 20 Potrošnja sklopovskih resursa cv::cvtColor metode

Na lijevom grafu vidimo ukupno sistemsko opterećenje, gdje je udio aplikacije označen narančastom linijom, a desno je prikazana potrošnja po individualnim jezgrama procesora, gdje tamniji dio grafa čine sistemski pozivi, a svjetliji predstavlja ukupno opterećenje. Crvenim linijama označen je trenutak kada je aplikacija pokrenuta. Lijevi graf pokazuje da upravo naša aplikacija troši gotovo sve CPU i mrežne resurse, dok pomoću desnog grafa vidimo da je iskorišteno svih 16 logičkih jezgri. Profiliranje programa pokazuje da

OpenCV paralelizaciju postiže automatskim korištenjem Tbb biblioteke, odnosno posao je particioniran bez da smo mi to morali ručno obaviti. Daljnje promatranje vodi do `cv::bayer2RGB` funkcije, a ispod nje se vide SSE primitivi. Drugim riječima, OpenCV samostalno zna iskoristiti sve dostupne računalne resurse i stoga bilo kakvi pokušaji ručnog unaprijeđena samo dodaju više posla, bilo u smislu raspoređivanja zadataka ili rastavljanja i sastavljanja slike, kvare performanse te su zato napušteni.

Što se tiče pokušaja korištenja GPU, promatranje iskorištenja sklopovlja i ovdje otkriva usko grlo, što prikazuje slika 21.



Slika 21 Potrošnja GPU resursa `cvt::Color` metode

Gornji graf na slici je inačica aplikacije koja za debayering koristi GPU, dok je na donjem korišten samo CPU. Crvene linije opet predstavljaju početak rada aplikacije. Vidimo da se GPU koristi u oba slučaja, no uzrok toga je iscrtavanje slika na ekran, što se izvodi preko GPU. Kada GPU iskoristimo i za debayering, prosječna potrošnja naraste za samo 2%, ali sada znatnu količinu vremena trošimo na memorijske operacije i to u konačnici vodi do slabijih performansi. Treba imati na umu da ovdje opisan pristup korištenja GPU nije nužno optimalan te da bi specijalizirani OpenCL kôd, pogotovo ako bi uspio povezati dekodiranje, debayering i prikaz u jedan niz, bez potrebe višestrukog prebacivanja podataka na GPU, mogao pružiti znatno bolje performanse. No, ovdje smo već polako izašli iz opsega rada stoga daljnji naponi nisu ulagani u tom smjeru, već je odlučeno koristiti `cv::cvtColor` za debayering.

10.2.5.3 Korekcije i prikaz slike

Proces debayeringa je po svojoj prirodi unosi nove informacije u sliku kroz interpolaciju, što znači da njegov rezultat neće uvijek odgovarati izvornoj slici. To je posebice slučaj kad slika prolazi kroz proces kompresije s gubitcima, kao kod nas. Rezultat toga je da na prikazanim slikama primijećeno suviše zelene boje te su slike nešto tamnije nego je očekivano, pa je dodan opcionalni korak prije prikaza slike u kojem se umanjuju vrijednosti zelenog kanala, a svjetlina i kontrast slike povećaju. Ova opcija nije korištena u testovima kašnjenja jer vizualna kvaliteta slike nije strogi zahtjev na sustav, ali se ipak smatra dovoljno bitnom da ovakva funkcionalnost bude implementirana.

Prikaz slike je jednostavan po pitanju implementacije i sastoji se poziva `cv::imshow` metoda koja sliku pripremi za prikaz te poziva `cv::waitKey` metoda koja osvježi ekran. Te dvije metode zajedno čine oko 3,8% iskorištenih CPU resursa, no, i kao što je bio slučaj kod dekodiranja slike, nemamo konkretnih opcija za njihovo ubrzanje.

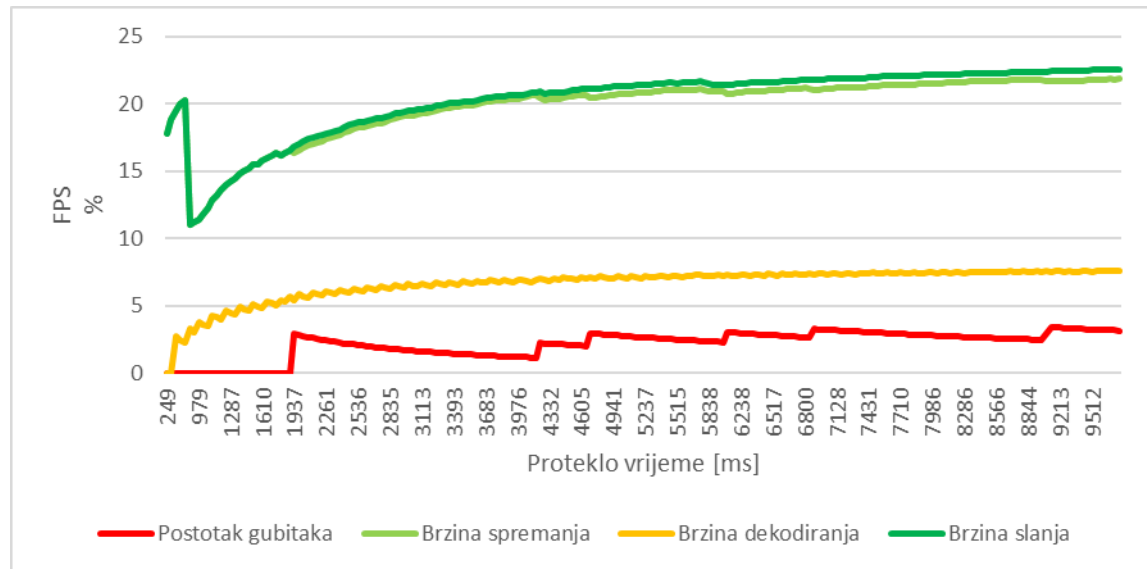
10.3 Usporedba s prvim prototipom

Nakon što je aplikacija optimizirana koliko je bilo razumno moguće, izvedeni su testovi kašnjenja s kraja na kraj. Jedan od većih nedostataka prvog prototipa je što na sličicama nije provodio debayering. To je učinjeno jer su inicijalne performanse bile slabije nego je poželjno i jer je prilikom formiranja video toka provedena H.264 kompresija zbog koje su vrijednosti piksela osjetno promijenjene pa pokušaji debayeringa takvog videa nisu davali vizualno zadovoljavajuće rezultate. Taj je problem otklonjen u drugom prototipu, no u svrhu pravedne usporedbe, izvedeni su testovi s i bez debayeringa.

Kao i kod prvog prototipa SPS, mjerenja kašnjenja inicijalno je bilo izvedeno uz pomoć eksterne kamere koja snima 240 FPS te brojanjem sličica između događaja u sceni, ispred kamere, te pojave istog na ekranu, a potom se iz broja sličica i perioda svake lako dobije ukupno kašnjenje. No, u tom su procesu primijećena tri fenomena: sustav se ponekad kratkotrajno zamrzne, brzina sustava nije konstanta kroz cijelu duljinu mjerenja te varira ovisno o sceni koju kamera snima.

Varirajuća brzina je tranzijentna pojava, djelomično objašnjenja uspostavljanjem veze, zauzimanjem i inicijalizacijom memorijskog prostora te drugim akcijama koje se obavljaju prije nego krene samo slanje slika, a dijelom radi načina kako se mjere performanse: ne uzima se pomični prosjek nego se apsolutni broj slika dijeli s ukupnim periodom rada. To

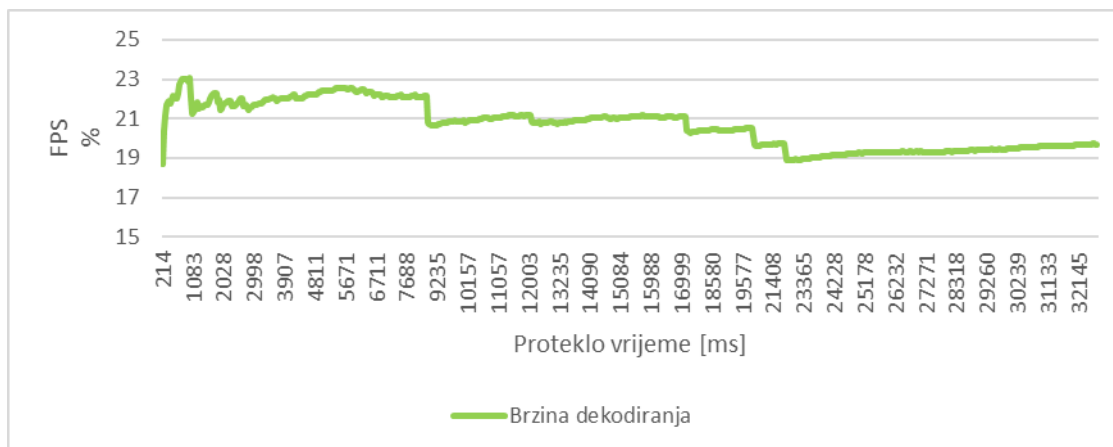
dvoje rezultira ponašanjem koje se može vidjeti na vremenskom grafu performansi sustava prikazanom na slici 22.



Slika 22 Vremenski odaziv sustava

Ovo je otežava testiranje jer zahtjeva da pričekamo dok sustav uđe u stabilno stanje, što nije bilo lako radi sljedećeg problema.

Zamrzavanje i povremeno rušenje sustava vrlo je vjerojatno posljedica složenosti scene i ispada paketa. Naime, kada dođe do ispada paketa, aplikacija zamjeni njegov sadržaj s unaprijed definiranom vrijednošću, primjerice nulama. Nakon toga će OpenCV funkcija dekodiranja pokušati rekonstruirati sliku, za što je, ako se radi o dovoljno velikim slikama, potrebna je povećana količina posla. Nadalje, ako je izgubljeno dovoljno podataka, ili barem podaci ključni za rekonstrukciju, funkcija dekodiranja jednostavno baci grešku i prekine s radom. U konačnici uzrokuje nagle padove brzine rada, što se može vidjeti na sljedećem grafu.



Slika 23 Vremenski odaziv sustava uz značajan gubitak paketa

Prikazani primjer predstavlja slučaj gori od prosječnog, no ilustrira zastoje upravo u periodima kada sustav dostiže stabilno stanje, zbog čega testovi traju dugo i rezultati imaju veću prosječnu devijaciju nego je očekivano.

Treći fenomen je ovisnost brzine sustava o složenosti scene koja se snima, što je posljedica kompresije. Naime, efikasnost JPEG kompresije proizlazi iz prostorne redundantnosti unutar slike, što primjerice znači da će se potpuno crna slika odlično komprimirati, a bijeli šum jako loše [50]. To je i testirano na 100 iteracija kompresije 4K slike i pokazalo se da bijeli šum zahtjeva 67,505 ms za kodiranje i dekodiranje te u prosjeku zauzima 4,781 MB, dok potpuno crna slika treba samo 9,705 ms za kodiranje i dekodiranje i u zauzima 0,093 MB. Drugim riječima, duljine obrade se razlikuju za faktor 6,956, a veličine za faktor 51,397 i posljedično varira brzina sustava.

Kako bi se osigurala ponovljivost testiranja, ove je faktore trebalo minimizirati. Zbog toga je napravljen drugi set testova gdje je ulaz u sustav bio unaprijed pripremljen niz slika, čime je složenost scene fiksirana. Odrađen je i veći broj mjerenja čime se minimizirao utjecaj početne nestabilnosti sustava. Inicijalna ideja bila je slati ponavljajući niz crvenih i plavih slika te napisati funkciju koja detektira kada dođe do promjene boje i na ekran ispiše trenutno vrijeme. Razlika trenutka promjene na klijentu i poslužitelju tada predstavlja ukupno kašnjenje. Upravo je zbog toga bilo potrebno učiniti prije spomenuto usklađivanje satova poslužitelja i klijenta, jer inače ne bismo mogli uspoređivati dobivena vremena. U obzir je uzeto i kašnjenje kamere, odnosno njegov nedostatak. Mjerenje je obavljeno istim principom kao i za cijeli sustav: eksternom kamerom je snimana scena s 240 FPS, i uspoređeno je trenutak promjene vanjske scene s trenutkom kada je prva sličica s tom promjenom ušla u program i razlika nikad nije prešla 10 ms, što je u većini testova unutar

prosječne devijacije, ali je i dalje uračunato u rezultate. Treći faktor vrijedan spomena je kašnjenje monitora, točnije vrijeme odaziva piksela monitora, koje je i više nego primjetno, a iznosi oko 8,32 ms u tranzicijama svjetlo u tamno i oko 36 ms u tranzicijama tamno u svjetlo te je korišten prosjek te dvije vrijednosti. Konačna kašnjenja prikazana su u tablici K.

Tablica 8 Usporedba kašnjenja SPS prototipa 1 i 2

	FPS Poslužitelja	FPS Primanja	FPS Dekodiranja	Kašnjenje [ms]	Prosječna devijacija [ms]
SPS1	30,000	30,000	1891,660	-	-
	20,000	20,000	170,830	-	-
	10,000	10,000	166,660	-	-
SPS2 No JPEG	31,382	12,850	12,830	37,061	4,017
SPS2	20,490	20,210	19,530	63,194	14,583
	54,930	54,490	23,580	61,574	9,362
	114,230	113,010	20,600	55,833	11,000
SPS2 Debayer	25,320	23,770	10,000	159,167	26,000
	54,320	50,120	10,120	191,204	28,395
	110,100	100,920	8,720	154,583	32,917

10.4 Poteškoće mjerenja

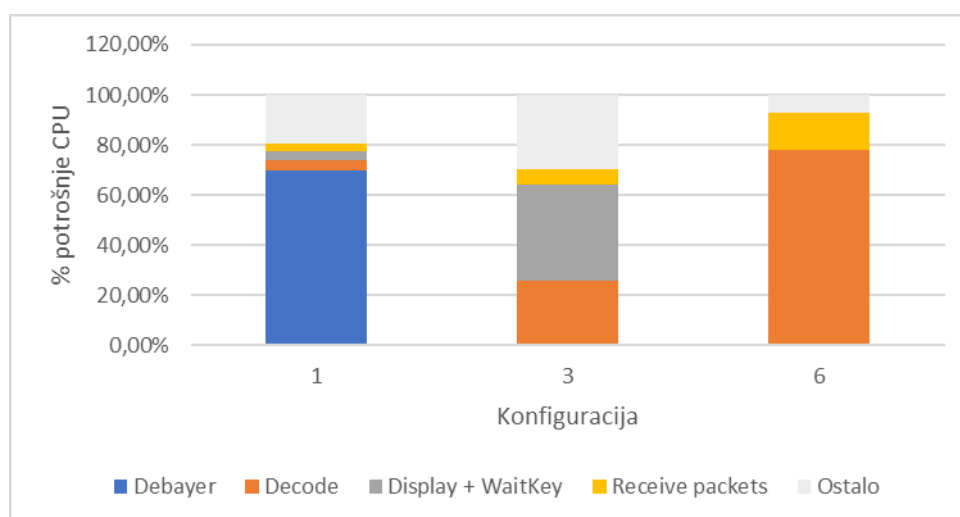
Gledajući dobivene rezultate, za SPS2 Debayer primjer možemo primijetiti jednu zanimljivost. Testovi su izvođeni pri 3 brzine slanja slika, odnosno tri brzine poslužitelja: 25, 54 i 110 FPS, što je moguće jer ne koristimo kameru nego unaprijed pripremljene slike, pa njihovo kodiranje možemo napraviti unaprijed i time vidjeti kakav bi utjecaj na konačno kašnjenje imali brža kamera, odnosno poslužitelj. Očekivano je da će uz porast brzine kamere i slanja ukupno kašnjenje padati jer, pod pretpostavkom da klijent uspijeva primati slike brzinom kojom ih poslužitelj šalje, „promjena u sceni“ će se brže propagirati kroz sustav i prije prikazana. Rezultati to i pokazuju, osim u primjeru kada SPS2 izvodi debayering.

Jedno od objašnjenja kako se to može dogoditi glasi ovako: ako je „promjena“, u našem slučaju period trajanja boje „A“, traje dovoljno kratko, odnosno ako je obrada slike dovoljno spora, može se dogoditi da promjena završi prije nego ju je klijent imao vremena

obraditi i prikazati. Teorem o uzorkovanju govori nam da bi frekvencija uzorkovanja signala, što odgovara frekvenciji prikazivanja, trebala biti barem dvostruko veća od maksimalne frekvencije signala, što odgovara frekvenciji slanja, no ovo se u praksi nije uspjelo postići. Spuštanjem rezolucije na 2560 x 1440 i gašenjem debayeringa, klijent maksimalno dostiže 28 FPS, a u prosjeku i manje. Daljnjim gašenjem prikaza slike postizemo 40,73 FPS, što i dalje nije dosta, a nije ni odviše korisna informacija s obzirom na to da je prikaz slike jedan od zahtjeva na sustav. Ipak je izvedeno profiliranje kôda s i bez korištenja raznih „ubrzanja“, a rezultati su prikazani u tablici 9 i slici N.

Tablica 9 Raspored potrošnje CPU po funkcijama klijenta u ovisnosti o omogućenim funkcionalnostima

Konfiguracija	Display	Debayer	Decode	FPS	Postotak potrošnje				
					Debayer	Decode	Display + WaitKey	Receive packets	Ostalo
1	da	da	da	10,00	69,66%	4,10%	3,80%	3%	19,44%
3	da	ne	da	28,83	0%	26%	38%	5,70%	29,90%
6	ne	ne	da	40,73	0%	78%	0%	15%	7,00%
5	ne	da	da	18,00	-	-	-	-	-
2	da	da	ne	15,73	-	-	-	-	-
4	da	ne	ne	32,55	-	-	-	-	-
7	ne	ne	ne	31,90	-	-	-	-	-

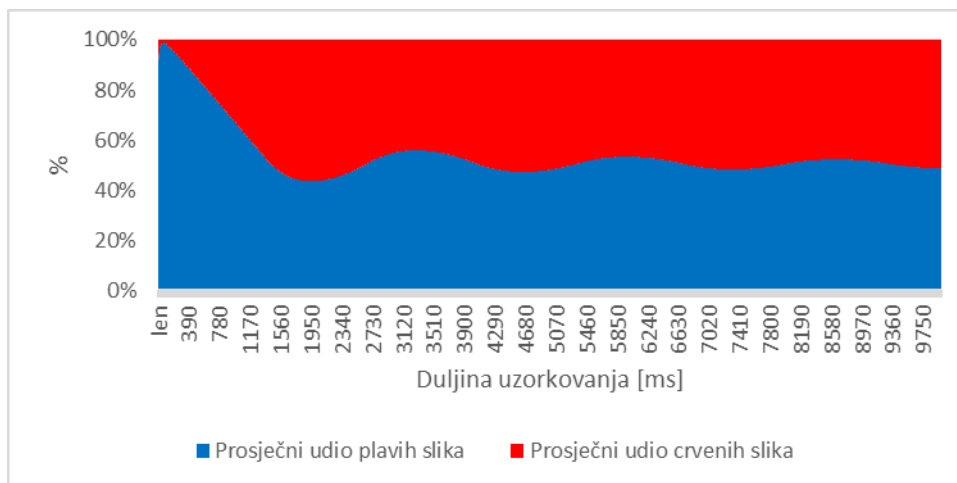


Slika 24 Grafički prikaz potrošnje CPU po funkcijama za odabrane konfiguracije

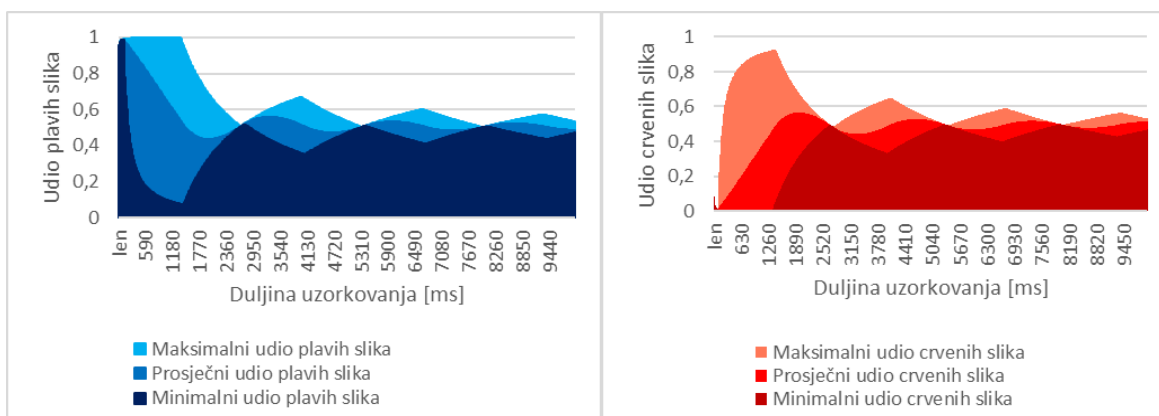
U konfiguracijama 1, 3 i 6 redom su uklonjene funkcije koje troše najviše CPU resursa, i time su postignuti gore spomenuti rezultati. U konfiguraciji 5 utvrđeno je da iako prikaz slike ima osjetnu cijenu, ona i dalje nije ni blizu cijene debayeringa. U konfiguracijama 2, 4 i 7 testirano su mogućnosti sustava ako se ne koristi kodiranje, no brzo dolazimo do problema zbog koje je kodiranje inicijalno i uvedeno: nekomprimirana 2560 x 1440 slika s 8 bita po pikselu ima 28,125 Mb podataka, odnosno mreža izmjerenog kapaciteta od 900 Mb/s može podržati maksimalno 32 FPS. Za 4K slike, odnosno prostornu rezoluciju 3840 x 2160, taj broj pada na 14,22 FPS. Zato nakon uklanjanja kodiranja i debayeringa prikaz slike ili nedostatak istog ne čini razliku u performansama klijenta: klijent prikazuje sve slike koje dolaze do njega, odnosno koje je mreža u mogućnosti isporučiti.

Zaključno, čak i bez debayeringa, prikaza slike i spuštanjem rezolucije na 2560 x 1440, klijent ne može doseći performanse poslužitelja, kojeg je jedino bilo i moguće natjerati na brži rad kodiranjem slika unaprijed, što naravno nije moguće kod klijenta. Posljedica toga je da, paradoksalno, brži rad poslužitelja može dati privid većeg kašnjenja.

Na uspješnost mjerenja ukupnog kašnjenja ne utječu samo brzina poslužitelja i klijenta, već i fazni pomak uzimanja uzoraka. Podesimo li primjerice da poslužitelj šalje 10 slika u sekundi, 5 plavih, 5 crvenih, svaka s periodom 100 ms. Ako klijent „uzima uzorak“ svakih 500 ms i krene s radom u 1. milisekundi, svaki uzorak će biti plavi. Ako frekvencija prikazivanja i slanja nisu višekratnici, izmjenično će se mijenjati dulji i kraći periodi plavih i crvenih slika. Ako frekvencije nisu višekratnici i uzorkovanje radimo dovoljno dugo, očekujemo da će svaka boja predstavljati 50% prosječnih uzoraka, a to pokazuju i praktični eksperimenti čiji su rezultati na slikama A i B. Slika 25 prikazuje omjer crvenih i plavih uzoraka u ovisnosti o duljini mjerenja, a slika 26 omjer plavih i crvenih slika u ovisnosti o trenutku u kojem kreće mjerenje, tj. o faznom pomaku.



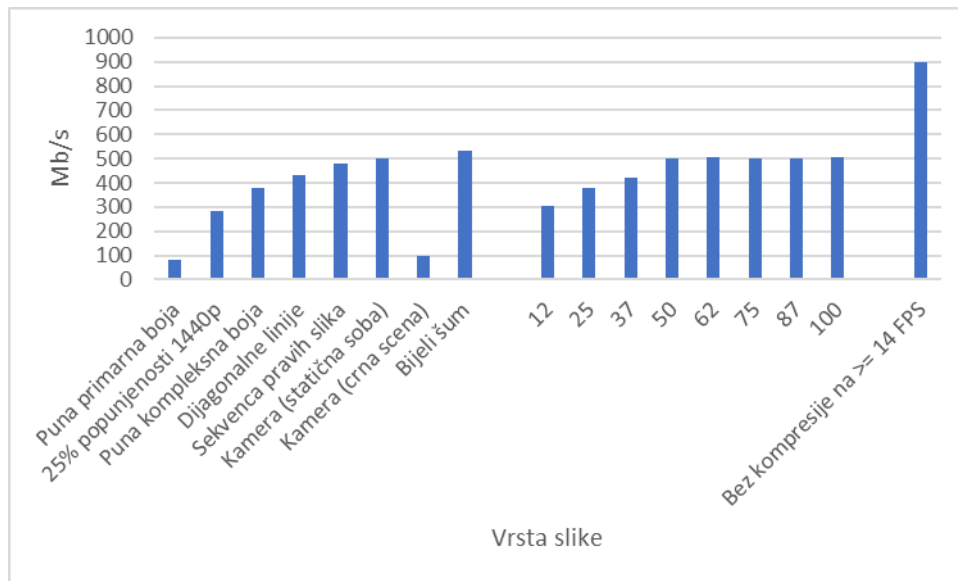
Slika 25 Omjer uzorka crvenih i plavih slika kroz vrijeme



Slika 26 Omjer uzorka plavih i crvenih slika u ovisnosti o faznom kutu uzorkovanja

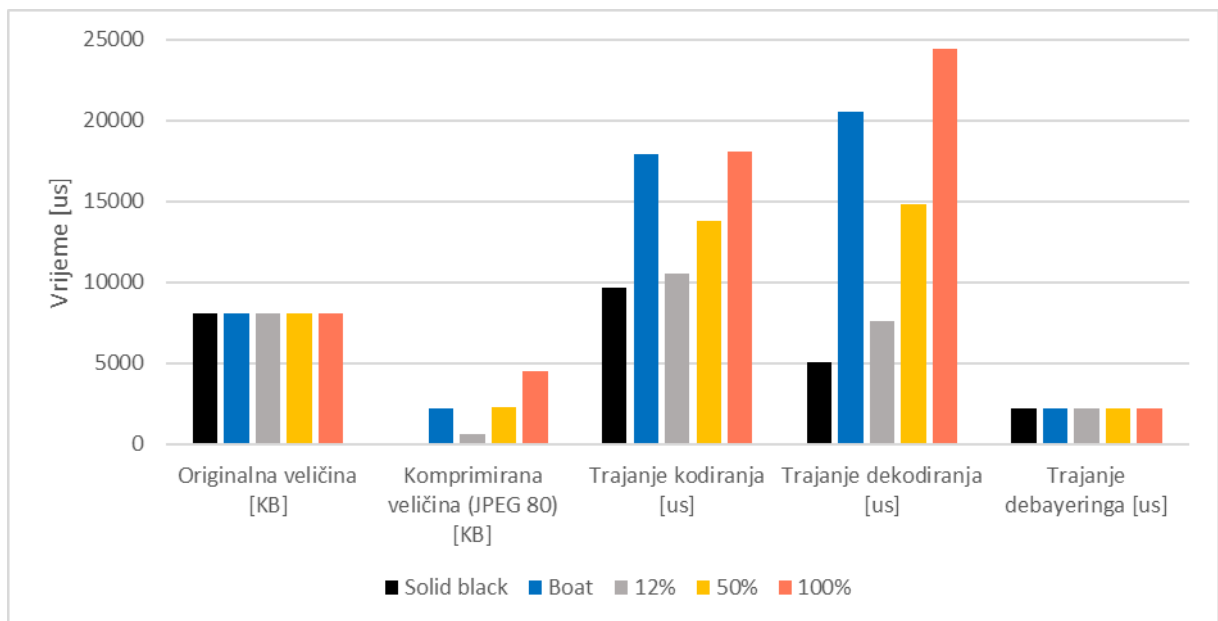
klijent ne može obraditi sve slike. Također, dok implementirani poslužitelj uspješno savladava izlaznu brzinu kamere, klijent bi imao koristi od daljnjih optimizacija.

Za kraj ćemo se na trenutak vratiti na izbor slike, jer i dalje ostaje neodgovoreno pitanje kako odabrati „referentni“ niz slika, ili scenu ako se koristi kamera, koji dobro predstavljaju potencijalnu stvarnu upotrebu sustava kada ista nije jasno definirana? Znanstveni radovi koji istražuju efikasnost i performanse koda često koriste dobro poznate slike kako bi se olakšala usporedba, dok testovi video koda znaju koristiti filmove u javnoj domeni, no ti primjeri mogu, ali i ne moraju biti reprezentativni buduće primjene ovog sustava. Mjerenja opterećenja mreže za razne nizove 4K slika, prikazana na slici 27, pokazuju da uz korištenje JPEG kodiranja s opcijom kvalitete 80, aplikacija ne troši više od 600 Mb/s, neovisno o vrsti slike.



Slika 27 Utjecaj složenosti scene na veličinu komprimirane slike

No, isto ne mora vrijediti za količinu vremena potrebnu za kodiranje različitih slika. Nadalje, iz profiliranja kôda znamo da je daleko najskuplja klijentska funkcija debayering stoga je za očekivati da kompleksnost slike ima osjetan utjecaj na brzinu obrade te kašnjenje s kraja na kraj. Stoga, ideja je iznose kašnjenja izmjeriti na nizu slika koji pokrivaju veći skup složenosti scena i time dobiti reprezentativniji uvid u ponašanje sustava u raznim okruženjima. U tu je svrhu generiran niz slika progresivno veće složenosti te su izmjerena vremena obrade i zauzeće mreže za njih te nekoliko drugih referentnih slika. Rezultati mjerenja prikazani su na slici 28, a primjeri referentnih i progresivno složenijih slika na slici 29.



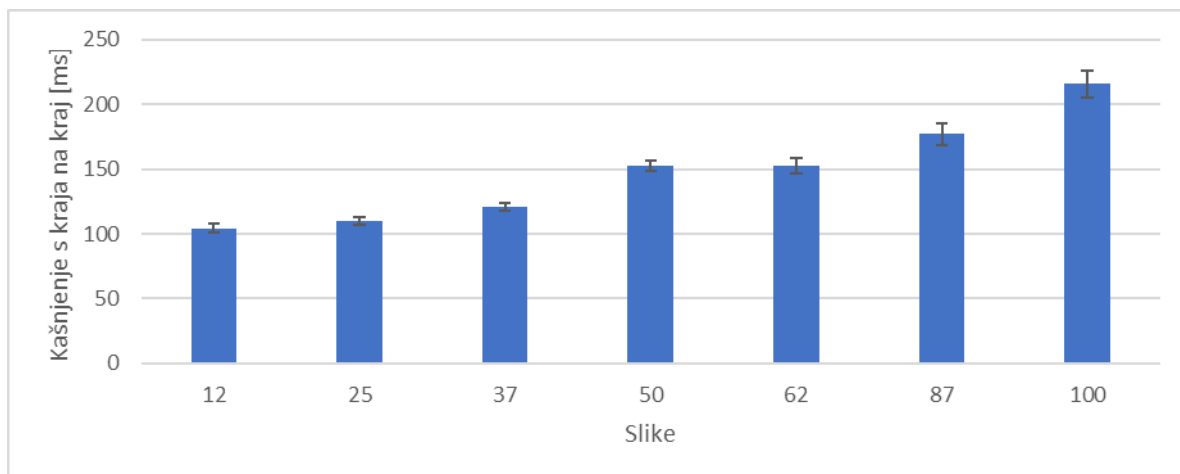
Slika 28 Utjecaj složenosti scene na veličinu komprimirane slike te trajanje obrade



Slika 29 Primjeri referentnih i progresivno složenijih slika

Interesantno je vidjeti kako kompleksnost slike znatno utječe na trajanje kodiranja i dekodiranja slika, ali zato nema gotovo nikakav utjecaj na trajanje debayeringa, što je suprotno od izvorne pretpostavke. Ovi rezultati također ukazuju na to da bi iduća inačica SPS-a mogla imati koristi od bržeg kodiranja i dekodiranja, potencijalno uz pomoć spomenutog grafičkog akceleratora.

Za kraj, nakon što su generirane slike uspoređene po potrebnom vremenu obrade i veličini, vidimo da one u rasponu od 50% do 100% popunjenosti podudaraju pravim scenama koje je kamera snimila i pretpostavka je da predstavljaju raspon zahtjeva koje se u budućnosti mogu očekivati od sustava. Njihova konačna kašnjenja s kraja na kraj prikazana su na slici 30.



Slika 30 Performanse sustava u ovisnost o složenosti

Zaključak

Postavljeni su teoretski temelji sustava za mrežni prijenos videa visoke rezolucije na kratke udaljenosti sa što kraćim kašnjenjem te su, nakon niza optimizacija, izmjerene mogućnosti dvije praktične implementacije istog, čime je ujedno i potvrđena njegova izvedivost. Prva implementacija imala je kašnjenje s kraja na kraj od 166,66 ms, a druga postiže 55,833 ms u istoj konfiguraciji. Druga implementacija također nudi dodatne mogućnosti, poput konverzije slike iz Bayer RGGB formata u RGB, no tada je kašnjenje 154,583 ms, te mogućnost slanja slika bez kompresije, pri čemu kašnjenje pada na 37,061 ms, što se može i dodatno smanjiti korištenjem bolje mrežne infrastrukture, reda veličine 2.5 do 10 Gigabita u sekundi. Analiza postavki rada otkrila je optimalne vrijednosti za minimiziranje kašnjenja, ali i moguće smjerove za napredak, od kojih se ističu dva: manje sklopovski zahtjevan algoritam kompresije te implementacija algoritama za dekompresiju i interpretaciju Bayer RGGB formata za grafički akcelerator. Ako postoji potreba za daljnjim ubrzanjem, tada bi arhitekture diskutirane u poglavlju 4.1 te sklopovlje prezentirano u poglavlju 6 i 7 mogli biti od interesa, a poglavlje 1.1 sadrži popis bežičnih tehnologija koje predstavljaju supstitut za bakreni medij korišten u prve dvije implementacije.

Literatura

- [1] C. Vieri, G. M. Lee, N. Balram, S. H. Jung, J. Yang, S. Y. Yoon, I. M. Kang. An 18 megapixel 4.3" 1443 ppi 120 Hz OLED display for wide field of view high acuity head mounted displays. *Journal of the Society for Information Display*, 26(5), 314–324., 2018. <https://doi.org/10.1002/jsid.658>
- [2] A. Juniper. Best FPV goggles for drone pilots in 2023. Digitalcameraworld. 2023. <https://www.digitalcameraworld.com/buying-guides/best-fpv-goggles> (Pristupljeno 25. svibnja 2023.).
- [3] "The Edge of the Cloud - 5G Technology Blog: The Components of Motion-to-Photon Latency in Virtual Reality Systems," *Edge-of-cloud*, 12. studenog, 2016. <http://edge-of-cloud.blogspot.com/2016/11/the-components-of-motion-to-photon.html> (Pristupljeno 12. siječnja 2022.).
- [4] "Understanding - and Reducing - Latency in Video Compression Systems," *Design-reuse*, 7. listopada, 2013. <https://www.design-reuse.com/articles/33005/understanding-latency-in-video-compression-systems.html> (Pristupljeno 12. siječnja 2022.).
- [5] "HDMI Technology: Specifications and Programs," *Hdmi*, <https://www.hdmi.org/spec/index> (Pristupljeno 12. siječnja 2022.).
- [6] "USB-IF Document Library," *USB-IF*, <https://www.usb.org/documents> (Pristupljeno 12. siječnja 2022.).
- [7] "Specifications | PCI-SIG," *Pcisig*, Jan. 1. siječnja, 2000. <https://pcisig.com/specifications/conventional/> (Pristupljeno 12. siječnja 2022.).
- [8] "EF - Canon Camera Museum," *Global.Canon*, 9. prosinca 2016. <https://global.canon/en/c-museum/product/film88.html> (Pristupljeno 12. siječnja 2022.).
- [9] "EF Lens Control - ISSI.", 21. ožujka 2017., <https://innssi.com/canon/> (Pristupljeno 12. siječnja 2022.).
- [10] "Wireless Lens Control Systems – all you need to know" , *Vmi*, 29. siječnja 2021, <https://vmi.tv/blog/learn-help/wireless-lens-control-systems-all-you-need-to-know/> (Pristupljeno 12. siječnja 2022.).

- [11] "JMS583 Product Brief," Jmicron, 15. svibnja 2018. https://www.jmicron.com/file/download/1012/JMS583_Product+Brief.pdf (Pristupljeno 12. siječnja 2022.).
- [12] "How Many USB Devices Can I Connect?", Acroname, 23. svibnja 2019., <https://acroname.com/blog/how-many-usb-devices-can-i-connect> (Pristupljeno 12. siječnja 2022.).
- [13] Y. Li, "Linux USB Driver", Yannik520, 13. kolovoza, 2016. <https://yannik520.github.io/usb/usb.html> (Pristupljeno 12. siječnja 2022.).
- [14] "AMD Summit Ridge SoC," Wikichip, 20. ožujka 2019. https://en.wikichip.org/w/images/a/a8/AMD_Summit_Ridge_SoC.svg (Pristupljeno 12. siječnja 2022.).
- [15] H. Hagedoorn, "Documents show AMD AM5 will use DDR5 and PCI-E 4.0, not 5.0," Guru3d, 21. kolovoza 2021. <https://www.guru3d.com/news-story/documents-show-amd-am5-will-use-ddr5-and-pci-e-4-0not-5.html> (Pristupljeno 12. siječnja 2022.).
- [16] "Make it Faster: More Throughput or Less Latency?", Ni, 28. kolovoza 2021. <https://www.ni.com/en-rs/innovations/white-papers/13/make-it-faster--more-throughput-or-less-latency-.html> (Pristupljeno 12. siječnja 2022.).
- [17] "3.9 WinDriver USB Architecture", Jungo, 1. veljače, 2002. https://www.jungo.com/st/support/documentation/windriver/802/wdusb_man_mhtml/node31.html (Pristupljeno 12. siječnja 2022.).
- [18] M. Hopkins, A. Viviano, "Architecture: USB host controller extension (UCX)," Docs, 17. siječnja, 2022. <https://docs.microsoft.com/en-us/windows-hardware/drivers/usbcon/get-started-with-host-controller-driver-development> (Pristupljeno 12. siječnja 2022.).
- [19] A. Thompson, C. Newburn, "GPUDirect Storage: A Direct Path Between Storage and GPU Memory | NVIDIA Technical Blog", Nvidia Developer, 6. kolovoza, 2019. <https://developer.nvidia.com/blog/gpudirect-storage/> (Pristupljeno 12. siječnja 2022.).
- [20] "NVIDIA GPUDirect Storage Overview Guide :: NVIDIA GPUDirect Storage Documentation," Docs, 11. svibnja, 2022. <https://docs.nvidia.com/gpudirect-storage/overview-guide/index.html> (Pristupljeno 12. siječnja 2022.).
- [21] F. Serthenko, A. Uvarov, "Full GPU Image Processing Pipeline for Camera Applications," Fastcompression,

https://www.fastcompression.com/pub/2014/S4151_Full_Image_Processing_Pipeline_on_the_GPU_for_Camera_Applications.pdf (Pristupljeno 12. siječnja 2022.).

[22] "GigE Vision for Realtime MV," Nstx, 11. svibnja, 2010. https://nstx.pppl.gov/nstxhome/DragNDrop/Operations/Diagnostics_&_Support_Sys/D1C_CD/GigE_Vision_for_Realtime_MV_11052010.pdf (Pristupljeno 12. siječnja 2022.).

[23] "video applications - Klotzfiber | KLOTZ AIS Fiber Optic," Klotzfiber, 21. siječnja, 2022. <https://www.klotzfiber.com/video-applications/> (Pristupljeno 12. siječnja 2022.).

[24] "USB-IF Document Library," USB-IF, <https://www.usb.org/documents> (Pristupljeno 12. siječnja 2022.).

[25] "Vision Standards - Camera Link HS Standard," Automate, <https://www.automate.org/a3-content/vision-standards-camera-link-hs> (Pristupljeno 12. siječnja 2022.).

[26] "CV-PT-HEAD Instructions," Marshall-usa, <https://marshall-usa.com/pdf/CV-PT-HEAD-Instructions.pdf> (Pristupljeno 12. siječnja 2022.).

[27] "Lens Control - ISSI," Innssi, 13. siječnja, 2022. <https://innssi.com/lens-control/> (Pristupljeno 12. siječnja 2022.).

[28] "Using LUCID's TFL to EF Lens Controller (Birger Adapter) | LUCID Support & Help," Support, 8. ožujka, 2021. <https://support.thinklucid.com/knowledgebase/using-lucids-tfl-to-ef-lens-controller/> (Pristupljeno 12. siječnja 2022.).

[29] S. F. Ray, Applied photographic optics : lenses and optical systems for photography, film, video, electronic and digital imaging. Oxford: Focal, 2002.

[30] "FUJINON Television Lenses," Fujifilm, 15. siječnja 2021. https://asset.fujifilm.com/master/global/files/2021-01/2aab23426dd3b55a2d28d8c512850833/210115_21_FUJINON_TV_EN.pdf (Pristupljeno 12. siječnja 2022.).

[31] M. Stensvold, "Image Stabilization," Outdoorphotographer, 3. ožujka, 2007. <https://www.outdoorphotographer.com/photography-gear/lenses/image-stabilization/> (Pristupljeno 12. siječnja 2022.).

- [32] J. Meyer, "Which cameras have IBIS?," Camerajabber, 24. siječnja, 2022. <https://camerajabber.com/buyersguides/which-cameras-have-ibis/> (Pristupljeno 12. siječnja 2022.).
- [33] V. Avramelos, G. Wallendaël, P. Lambert, "Real-Time Low-Complexity Digital Video Stabilization in the Compressed Domain," Ghent University, IDLab Technologiepark, pp. 1-5, <https://biblio.ugent.be/publication/8578339/file/8578341.pdf>.
- [34] "Instrument Bus Performance – Making Sense of Competing Bus Technologies for Instrument Control," Ni, 12. kolovoza, 2021. <https://www.ni.com/en-rs/innovations/white-papers/11/instrument-bus-performance---making-sense-of-competing-bus-techn.html> (Pristupljeno 12. siječnja 2022.).
- [35] "What Makes a Bus High Performance?," Ni, 6. svibnja, 2022. <https://www.ni.com/en-rs/innovations/white-papers/06/what-makes-a-bus-high-performance-.html> (Pristupljeno 12. siječnja 2022.).
- [36] "CE511-HN 4K HDMI 2.0 PCIe capture card," Braemac, 2016. http://www.braemac.co.uk/images/AVerMedia/PCIe/DS_CE511HN_EN_20160630.pdf (Pristupljeno 12. siječnja 2022.).
- [37] "EMS XtremeRGB-Ex1," lvision, 27. siječnja, 2022. https://www.lvision.co.il/pdfs/EMS/XtremeRGB-Ex1_datasheet.pdf (Pristupljeno 12. siječnja 2022.).
- [38] "VisionSC-UHD2," Datapath documents, 8. rujna, 2021. https://www.datapathdocuments.co.uk/wp-content/uploads/VisionSC-UHD2_datasheet_EN.pdf (Pristupljeno 12. siječnja 2022.).
- [39] "HDMI :: Resources :: FAQ," Archive, 2018. <https://web.archive.org/web/201802222200543/https://www.hdmi.org/learningcenter/faq.aspx> (Pristupljeno 12. siječnja 2022.).
- [40] D. Mayhew, V. Krishnan, "PCI express and advanced switching: evolutionary path to building next generation interconnects," 11th Symposium on High Performance Interconnects, 2003, pp. 21-29, doi: 10.1109/CONNECT.2003.1231473.
- [41] "Short FAQs about Pro Capture," Magewell, 19. siječnja 2021. <https://www.magewell.com/files/documents/FAQs/Short%20FAQs%20about%20Pro%20Capture.pdf> (Pristupljeno 12. siječnja 2022.).

- [42] "Pro Capture HDMI 4K Plus LT Tech Specs," Magewell, 2022. <https://www.magewell.com/tech-specs/pro-capture-hdmi-4k-plus-lt> (Pristupljeno 12. siječnja 2022.).
- [43] "NVIDIA GPUDirect for Video | NVIDIA Developer," Nvidia Developer, 2022. <https://developer.nvidia.com/gpudirectforvideo> (Pristupljeno 12. siječnja 2022.).
- [44] "ASI183 Manual V1.4," Astronomy-imaging-camera, listopad 2021. https://astronomy-imaging-camera.com/manuals/ASI183_Manual_EN_V1.4.pdf
- [45] „Everything you need to know about Astrophotography Pixel Binning (the fundamentals)“, lipanj 2020., <https://astronomy-imaging-camera.com/tutorials/everything-you-need-to-know-about-astrophotography-pixel-binning-the-fundamentals/>
- [46] "ASI183 Manual V1.4," Astronomy-imaging-camera, listopad 2021. https://astronomy-imaging-camera.com/manuals/ASI183_Manual_EN_V1.4.pdf
- [47] V. Bui, L. Chang, D. Li, L. Hsu, M. Y. Chen. Comparison of lossless video and image compression codecs for medical computed tomography datasets. 2016. <https://doi.org/10.1109/bigdata.2016.7841075>
- [48] Kcampbell. What Is A Hard Drive & The Difference Between HDD and SSD. Securis. 2023. <https://securis.com/news/what-is-a-hard-drive-and-the-difference-between-hdd-and-ssd/>
- [49] "Understanding - and Reducing - Latency in Video Compression Systems," Design-reuse, 7. listopada, 2013. <https://www.design-reuse.com/articles/33005/understanding-latency-in-video-compression-systems.html> (Pristupljeno 12. siječnja 2022.).
- [50] Prezentacije iz kolegija Videokomunikacijske tehnologije, Prof. dr. sc. Sonja Grgić
- [51] Gstreamer API reference. API reference, siječanj 2023. <https://gstreamer.freedesktop.org/documentation/libs.html?gi-language=c> (Pristupljeno 15. siječnja 2023.).
- [52] „What is the actual real-life speed of wireless networks ?“. SpeedGuide, siječanj 2023. <https://www.speedguide.net/faq/what-is-the-actual-real-life-speed-of-wireless-374> (Pristupljeno 15. siječnja 2023.).
- [53] „Comparative Performance Analysis of Wireless Communication Protocols for Intelligent Sensors and Their Applications“, IJACSA, Vol. 5, No. 4, 2014.

- [54] „How To Reduce Lag - A Guide To Better System Latency“, NVIDIA, listopad 2020. <https://www.nvidia.com/en-us/geforce/guides/system-latency-optimization-guide/> (Pristupljeno 15. siječnja 2023.).
- [55] R. Langseth, V. R. Gaddam, H. K. Stensland, C. Griwodz, P. Halvorsen. An Evaluation of Debayering Algorithms on GPU for Real-Time Panoramic Video Recording. 2014
- [56] S. Dhawan, A Review of Image Compression and Comparison of its Algorithms, Kurukshetra University, IJECT, 2(1), <http://iject.org/vol2issue1/sachindhawan.pdf>
- [57] M. Shabunin, „Opencv3 Transparent API“, OpenCV Wiki, prosinac 2017. <https://github.com/opencv/opencv/wiki/Opencv3> (Pristupljeno 26. travnja 2023.).
- [58] T. Savvas, Algebraic Modeling of Transformations from Bayer to RGB Images, Technical University of Crete, 2006.

Sažetak

Sustav za prenošenje 4K video snimke s niskom latencijom na male udaljenosti

Ovaj rad istražuje ključne aspekte sustava za prijenos videa s malim kašnjenjem preko LAN mreže, što uključuje teoretska i praktična razmatranja spajanja i prijenosa video signala s 4K kamere u računalo, kodiranja tog signala prema mogućnostima lokalne mreže, prijenos preko mreže s minimalnim potrebnim kašnjenjem, primanje, dekodiranje i obrada podataka i konačno iscrtavanje na ekran. Osmišljena je arhitektura sustava i testirane dvije implementacije te se cijelo vrijeme vodilo računa o optimalnim postavkama kamere, kodiranja, prijenosa i dekodiranja s ciljem minimizacije kašnjenja. Sustav je optimiziran u skladu s novim saznanjima te su predstavljene konačne mogućnosti i kašnjenja.

Ključne riječi: Video, 4K, prijenos, u živo, strujanje, LAN, kašnjenje, latencija, kodiranje, kompresija, OpenCV, Gstreamer, H.264, JPEG, RTSP, RTP, UDP

Summary

A system for transmitting 4K video footage with low latency over short distances

This paper explores key aspects of a system for transmitting video over a LAN network with low latency, including theoretical and practical considerations of connecting and transmitting video signals from a 4K camera to a computer, encoding the signal based on the capabilities of the local network, transmitting over the network with minimal required latency, receiving, decoding, and processing the data, and finally rendering it on the screen. An architecture for the system has been designed, and two implementations have been tested, taking into account optimal camera, encoding, transmission, and decoding settings to minimize latency. The system has been optimized based on new findings, and the final capabilities and latencies are presented.

Keywords: Video, 4K, transmission, live streaming, LAN, latency, encoding, compression, OpenCV, Gstreamer, H.264, JPEG, RTSP, RTP, UDP